Deep Learning on RISC-V Platforms at the Edge: A Perspective on the Hardware and Software Support

GIOVANNI AGOSTA, Dipartimento di Elettronica, Informazione e Bioingegneria (DEIB), Politecnico di Milano, Milano, Italy
ANDREA GALIMBERTI, Dipartimento di Elettronica, Informazione e Bioingegneria (DEIB), Politecnico di Milano, Milano, Italy
DAVIDE ZONI, Dipartimento di Elettronica, Informazione e Bioingegneria (DEIB), Politecnico di Milano, Milano, Italy

The growing demand for always-on intelligence in resource-constrained devices makes edge deployment of deep learning both a necessity and a challenge, requiring platforms that combine efficiency, scalability, and flexibility. RISC-V has emerged as the de facto standard architecture for modern computing platforms at the edge tasked with deep-learning workloads, a trend reinforced by the increasing availability of commercial solutions tailored for inference. This survey delivers a structured taxonomy of the hardware architectures for deep learning at the edge, classified according to how they process data in parallel, represent data, and optimize data movement and whether they implement an application-specific design, and of the supporting software tools, ranging from hardware-software co-design approaches to autotuning and compiler frameworks. Finally, it identifies a set of key findings and outlines the most promising directions for research in the field.

CCS Concepts: • **Computer systems organization** → **Embedded systems**; • **Computing methodologies** → **Machine learning approaches**; • **General and reference** → *Surveys and overviews;*

Additional Key Words and Phrases: RISC-V, deep learning, deep neural networks, edge computing, hardware architectures, data representation, memory optimization, hardware-software co-design, autotuning, compilers

ACM Reference Format:
Giovanni Agosta, Andrea Galimberti, and Davide Zoni. 2025. Deep Learning on RISC-V Platforms at the Edge: A Perspective on the Hardware and Software Support. *ACM Comput. Surv.* 58, 5, Article 121 (November 2025), 37 pages. https://doi.org/10.1145/3772277

1 Introduction

Deep learning (DL) is a subset of **machine learning (ML)** that involves algorithms known as **artificial neural networks (ANNs)**, inspired by the structure and function of the human brain. ANNs consist of interconnected nodes, the so-called neurons, and are designed to recognize patterns

Giovanni Agosta, Andrea Galimberti, and Davide Zoni contributed equally; author names are listed in alphabetical order. This work was partially supported by the European Union's Horizon Europe Chips JU project ISOLDE (Grant No. 101112274).
Authors' Contact Information: Giovanni Agosta, Dipartimento di Elettronica, Informazione e Bioingegneria (DEIB), Politecnico di Milano, Milano, Italy; e-mail: giovanni.agosta@polimi.it; Andrea Galimberti (corresponding author), Dipartimento di Elettronica, Informazione e Bioingegneria (DEIB), Politecnico di Milano, Milano, Italy; e-mail: andrea.galimberti@polimi.it; Davide Zoni, Dipartimento di Elettronica, Informazione e Bioingegneria (DEIB), Politecnico di Milano, Milano, Italy; e-mail: davide.zoni@polimi.it.

ACM 0360-0300/2025/11-ART121
https://doi.org/10.1145/3772277

and make decisions by processing input data [62]. In particular, "deep" refers to the use of multiple so-called hidden layers in the neural network between the input and output ones, enabling the model to learn and represent data with increasing levels of abstraction [151]. DL's capabilities have revolutionized many fields by providing state-of-the-art performance in tasks such as text and speech translation [171], medical imaging [155], and computational chemistry [61].

As DL applications grow in complexity and increasingly involve private and sensitive data, the need to process data locally has led to the emergence of the edge computing paradigm [147]. This paradigm reduces latency, energy consumption, and communication costs while improving security. Recent trends push both training, i.e., learning patterns from data by adjusting a model's parameters, and inference, i.e., using a trained model to make predictions, towards the edge in applications ranging from obstacle avoidance in self-driving cars [103] to autonomous flight of unmanned aerial vehicles [126]. Training at the edge preserves privacy by avoiding the transfer of raw data to centralized servers, while inference benefits from low latency and improved quality of service by operating close to sensors, actuators, and end users.

The complexity of DL applications is in stark contrast with the constrained nature of devices at the edge, whether in terms of area, computational capabilities, cost, or energy and power consumption [196]. Deploying such complex applications on low-power, resource-constrained platforms such as **microcontroller units (MCUs)** requires therefore a careful optimization and co-design of the computing platform both at the hardware [157] and software [102] levels, and of the implemented DL models [28].

The RISC-V **instruction set architecture (ISA)** has emerged as the de facto standard architecture for modern edge computing platforms, encompassing both academic and commercial ones. RISC-V, that implements a **reduced instruction set computer (RISC)** architecture, is distributed as an open standard and is specifically designed with the goal of providing modularity and extensibility to computer designers [63]. Modularity is achieved by a minimal baseline instruction set, meant to be sufficient to form a compiler target and to support modern operating system environments and containing solely 40 instructions, whereas separate ISA extensions are each devoted to a specific set of additional features. Extensibility is granted by the unused opcode space, which can be exploited to implement custom instructions and ISA extensions according to the specific needs of computer designers, as well as by the open-source and royalty-free license.

1.1 Existing Surveys

The topics of DL, edge computing, and RISC-V architectures have been extensively surveyed in the open literature. Existing surveys can be grouped into three main categories.

First, several works focus on RISC-V, including comparisons of **central processing unit (CPU)** cores targeting **field-programmable gate arrays (FPGAs)** [76], application-class cores [45], and resource-constrained ones [51], together with ISA extensions for **artificial intelligence (AI)** workloads [39] and available software support [83, 115].

Second, a large body of surveys addresses DL and ML at the edge more broadly, covering TinyML [22], distributed training and inference [28], deployment on MCUs and smartphones [19, 182], and tools and frameworks for edge ML [121]. Related surveys review compression and optimization techniques for **deep neural networks (DNNs)** [33], with emphasis on their suitability for hardware acceleration [42, 175], and provide taxonomies of **hardware-software (HW/SW)** co-design methods and accelerator-oriented optimizations for efficient edge inference [2, 80, 157].

Third, a significant number of surveys analyze hardware accelerators for DL, including FPGA- and ASIC-based implementations [64, 120], CNN-oriented accelerators [23, 145, 164], precision-scalable and systolic-array designs [21, 183], and accelerators for heterogeneous high-performance

platforms [158]. Security-oriented surveys complement these with analyses of threats and countermeasures for DL accelerators [117].

While prior surveys provide valuable overviews of RISC-V cores, DL accelerators, or general edge ML, none offers a unified perspective that spans both the hardware and software stacks required for deploying DL workloads on RISC-V platforms at the edge. This gap motivates the present work.

1.2 Contributions

RISC-V is nowadays widely adopted as the primary architecture for contemporary edge computing systems, including those tasked with DL workloads. Concrete industrial deployments highlight the growing adoption and maturity of RISC-V architectures tailored for efficient DL inference in resource-constrained environments. Notable examples include the *GreenWaves GAP9* **system-on-chip (SoC)**, which integrates a RISC-V multi-core cluster with an ultra-low-power accelerator for always-on AI tasks; *SiFive*'s *Intelligence* cores, offering scalable RISC-V-based solutions for efficient inference across embedded and edge scenarios; *Alibaba*'s *XuanTie* processors, which combine high-performance RISC-V CPUs with AI-focused extensions; and *SemiDynamics*' portfolio of solutions for AI processing, including vector and tensor units.

Dealing with DL applications at the edge exacerbates the need to rigorously design such RISC-V computing platforms to satisfy the performance requirements within the energy, power, and area constraints, mandating the need for optimizing both the hardware and software aspects. Conversely, the RISC-V ecosystem includes a broad range of solutions, due to the open standard nature of the RISC-V architecture, with specific optimizations at the hardware and software levels to maximize their efficiency, thus making it paramount to carefully analyze the existing state of the art.

This manuscript aims to deliver a comprehensive analysis of how to effectively design modern RISC-V-based computing platforms on which to deploy DL applications at the edge, that is, not provided by any survey from the open literature, to the best of our knowledge.

In particular, this survey delivers three main contributions:

(1) it provides a holistic view of the existing solutions from the open literature, encompassing both the hardware-side and software-side aspects and ranging from microarchitectural and architectural ones up to those operating at the software and compiler level;
(2) it identifies a set of key findings concerning what is available and what is lacking in the RISC-V ecosystem of solutions meant for deploying DL applications at the edge;
(3) it outlines the most promising directions for future research on the topic, with the goal of fostering further work from the consortiums and stakeholders in the RISC-V ecosystem, ranging from experts in the DL and compiler domains to hardware and system designers.

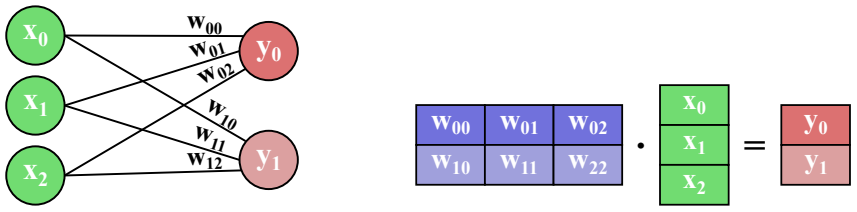
1.3 Manuscript Organization

The rest of this survey manuscript is organized into four sections and two appendices.

Section 2 outlines the essential background on the main DL models and optimization strategies. Sections 3 and 4 classify the RISC-V hardware architectures for DL at the edge and the supporting software tools from the open literature, respectively. Finally, Section 5 summarizes our key findings and outlines the most promising directions for future research in the field. Appendices A and B provide an additional background on the RISC-V ISA and on data representation formats for DL.

2 Background on Deep Learning

This section discusses the essential background on DL, defining in Section 2.1 the most common classes of *DNNs*, discussing in Section 2.2 the basic optimization approaches that enable reducing the complexity of the DL models and thus ease their deployment on edge platforms, and outlining in Section 2.3 the computational characteristics of the various models.



(a) Dense layer (b) Matrix-vector multiplication

Fig. 1. Correspondence between dense layers in NNs and MVMs. (a) A dense layer with three input neurons and two output neurons, where each edge encodes a trainable weight. (b) The equivalent linear algebra representation, where the weight matrix is multiplied by the input vector to produce the output. This equivalence explains why dense and convolutional layers are ultimately expressed as matrix operations, thereby motivating the central role of optimizing MVM at the edge.

2.1 Model Families

CNNs are a subset of DNNs that are specifically designed for processing data with a grid-like topology, e.g., images and videos. They leverage convolutional layers that apply filters to input data, enabling the detection of local patterns and features [97].

Variants of CNNs include ***temporal convolutional networks (TCNs)***, which are optimized for sequence modeling tasks [96], and ***graph convolutional networks (GCNs)***, that operate instead on graph-structured data.

Recurrent neural networks (RNNs) possess connections that form directed cycles, unlike feedforward networks, enabling them to maintain a state and exhibit temporal dynamic behavior, thus making them well-suited for tasks involving time series data, such as language modeling and speech recognition [114].

Long short-term memories (LSTMs) are variants of RNNs that incorporate memory cells and gating mechanisms that regulate the information flow, thus allowing the network to capture long-range dependencies and maintain context over extended sequences [72].

Transformer networks emerged as the foundations to state-of-the-art natural language processing models such as *BERT* and *GPT* due to their capability to process input sequences in parallel, which can be traced back to their adoption of self-attention mechanisms to model dependencies between input and output elements [171].

2.2 Model Optimizations

Matrix multiplications remarkably take by far the largest share of execution time and memory accesses during both training and inference of DL models. Such matrix multiplications include both matrix-matrix multiplications, that are leveraged by self-attention mechanisms in Transformer networks, and **matrix-vector multiplications (MVMs)**, which are required by the convolutional and dense layers of DNNs, as depicted in Figure 1. Such drastic reliance on matrix multiplications can be traced back to the effective pairing between DL applications and **graphics processing units (GPUs)**, that have been consistently shown to effectively parallelize DL computations [89].

Reducing the size of a DL model, as well as the execution time of its training and inference, without significantly compromising its performance plays a crucial role in making it more efficient and practical to deploy, in particular in edge devices with limited memory and computational resources. So-called model compression methods include pruning, quantization, and more advanced techniques such as weight sharing, low-rank factorization, and using sparse matrices.

Pruning eliminates redundant or less significant weights and connections, not only decreasing the model's size but also enhancing its computational efficiency and helping mitigate overfitting.

Quantization involves reducing the precision of the weights and activations in a DNN, i.e., representing them with lower bit-widths, e.g., as 8-bit integers instead of 32-bit ones, decreasing the model's memory footprint and computational requirements and therefore speeding up inference and reducing energy and power consumption. ***Binary neural networks (BNNs)*** [75] and ***ternary neural networks (TNNs)*** represent extreme cases of quantization, where weights and activations are constrained to binary (0 and 1) and ternary (-1, 0, and 1) values, respectively.

Weight sharing involves reducing the number of unique parameters in a DNN by forcing multiple weights to share the same value, while *low-rank factorization* decomposes weight matrices into products of smaller matrices, significantly reducing the number of parameters. Inducing *sparsity*, i.e., employing matrices that contain a significant number of zero elements, is another effective approach that enables reducing both the computational and memory footprint of a DL model.

2.3 Computational Characteristics

Each DL model class exhibits distinct computational and memory behaviors that influence hardware and software design choices.

CNNs typically exhibit high spatial data reuse and regular memory access patterns, making them highly parallelizable across both data and operations. They benefit from architectures optimized for high throughput and local data reuse, and they are generally tolerant to reduced numerical precision, e.g., 8-bit fixed-point.

TCNs and RNNs, including LSTMs, introduce sequential dependencies across time steps, which limit the extent of parallelism that can be exploited. However, within each step, moderate parallelism can still be leveraged. These models often exhibit more irregular memory access than CNNs and can have varying sensitivity to quantization depending on the task.

GCNs deal with sparse, irregular graph data structures, leading to highly irregular memory access patterns and less predictable parallelism. Performance is often constrained by memory bandwidth and access latency, and they may require higher numerical precision to maintain model accuracy.

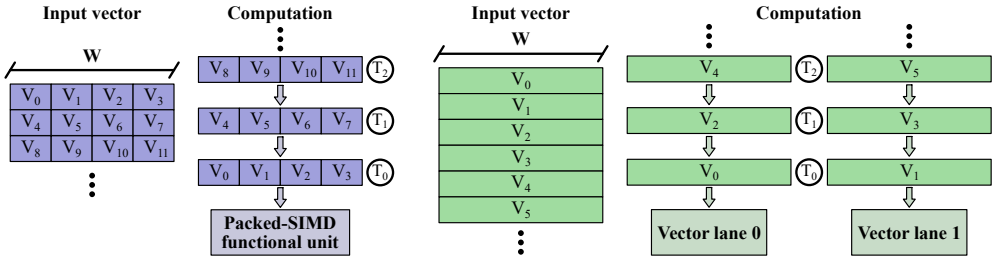
Transformer models, dominated by matrix multiplications in the attention and feedforward layers, expose substantial parallelism across both tokens and internal operations. They show relatively regular and dense memory access patterns and have demonstrated increasing tolerance to low-precision formats, such as *bfloat16* and even 4-bit to 8-bit integer representations.

Characterizing these properties allows mapping model demands to hardware features such as wide SIMD lanes, fast local memories, sparsity-aware compute units, and support for low-precision arithmetic, guiding the design of effective RISC-V-based edge solutions both on the hardware architecture side and on the software support one.

3 Hardware Architectures

Devising an architecture for DL at the edge entails a number of key design choices. On the one hand, there is a need to define how to exploit, at the hardware level, the parallelism, that is, intrinsic to DL applications. On the other hand, it is necessary to identify the number representation system and sizes of the selected format that best fit the contrasting requirements given by the accuracy of the model and by the performance, energy and power consumption, and area of the hardware computing platform. Moreover, the system designer must choose whether to lean towards a more general-purpose computing platform or implement instead an application-specific one, that is, custom-tailored to a specific DL model, class of models, or set of DL applications. Finally, optimizations more related to the memory side can further improve the energy efficiency and performance of DL applications, that typically require moving vast amounts of data.

This section discusses the RISC-V hardware architectures for DL at the edge from the open literature, outlining the main characteristics and solutions related to computation parallelism



(a) Packed-SIMD architecture (b) Vector architecture

Fig. 2. Comparison of monodimensional SIMD processing approaches. (a) In a packed-SIMD architecture, multiple sub-word elements are grouped within a machine word and processed in parallel by a single functional unit. (b) In a vector architecture, independent vector lanes process word-sized elements across longer vectors.

in Section 3.1, data representation in Section 3.2, application-specific design in Section 3.3, and memory-side optimization in Section 3.4.

3.1 Computation Parallelism

DL applications are particularly suited to parallel execution, thus defining how to parallelize the computation at the architectural level is a key decision in designing dedicated RISC-V-based systems. At a first glance, the primary choice in this respect is the traditional Flynn's taxonomy [47] partitioning into **single instruction, single data (SISD)**, **single instruction, multiple data (SIMD)**, and **multiple instruction, multiple data (MIMD)** architectures.

SISD. Scalar SISD processors, while at the lowest end of computing capabilities, can still provide support even for highly parallel tasks such as DL ones in resource-constrained scenarios due to their low cost and low power consumption.

SIMD. Matching the nature of DL computing tasks, that involve large MVMs, SIMD architectures are the most widely represented category. A first categorization of SIMD solutions is whether they involve a monodimensional or multidimensional (usually bidimensional) architecture. Monodimensional SIMD architectures, in Figure 2, are further categorized based on the scale of the array and of how it is processed. *Packed-SIMD* architectures operate on single-word vectors of sub-word elements, while *vector* ones operate on variable-length vectors of word-size elements and may process multiple of the latter at a time. Conversely, the bidimensional nature of SIMD architectures such as *systolic arrays* and ***in-memory computing (IMC)*** solutions matches effectively with the computation of MVMs, which are the most common kernel in DNN workloads.

MIMD. Multi-core architectures can handle multiple tasks simultaneously, thus improving overall performance by enabling multi-thread and multi-process execution, and may also be heterogeneous ones and include different types of cores, e.g., low-power and high-performance ones, to further optimize the energy efficiency of the computing platform. With the end of Moore's law and Dennard scaling, multi-core architectures have therefore become a widely adopted solution even at the edge.

3.1.1 SISD Architectures. The open literature provides a large set of RISC-V cores that provide the baseline scalar processing capabilities for fixed-point and **floating-point (FP)** arithmetic through hardware support for the I, M, F, and D standard ISA extensions.

The available RISC-V cores include resource-constrained ones such as 32-bit scalar in-order *RI5CY* [58] and *PicoRV32* [185], application-class ones such as 64-bit scalar in-order *Rocket* [100],

SHAKTI-C [54], and *CVA6* [187], and higher-performance 64-bit processors such as superscalar in-order *NOEL-V* [179] and speculative out-of-order *BOOM* [190].

These SISD cores offer essential scalar computation capabilities and are widely used in embedded and edge systems. However, their limited support for data-level parallelism makes them less suited for compute-intensive DL workloads. Some of the enhancements in Sections 3.2 and 3.3, such as the integration of optimized arithmetic units and of functional units tailored to DL-specific operations, are also applicable to SISD designs to improve inference performance and energy efficiency. In the rest of this section, we also provide quantitative comparisons, where available, to illustrate how more optimized solutions perform relative to baseline SISD designs in terms of throughput, energy efficiency, and hardware complexity.

3.1.2 Packed-SIMD Processors. The open literature provides multiple solutions that implement packed-SIMD processing, some of them specifically targeting the RISC-V P extension. The main ones are listed in Table 1, together with vector processors, systolic arrays, and IMC architectures.

The execution stage of the *RI5CY* RISC-V core [58] optionally implements support for packed-SIMD fixed-point and saturated arithmetic operations between 8-bit, 16-bit, and 32-bit operands, including dot-product instructions and unaligned memory access load-store ones. *RI5CY*'s **arithmetic logic unit (ALU)** also implements dedicated hardware support for the bit-manipulation instructions included in the Zbb extension of the RISC-V ISA, enabling effective processing of BNNs [40]. Binarization substitutes indeed MAC operations with simpler ones, in particular bit-wise **exclusive NOR (XNOR)** and population-count operations replace multiplications and accumulations, respectively. [87] similarly adds bit-manipulation support to *Rocket* [100].

MPIC [128] extends instead *RI5CY* with support for inference of mixed-precision **quantized neural networks (QNNs)**, i.e., neural networks with quantized weights and activations. *MPIC* targets 292 different combinations of operands at 2-bit, 4-bit, 8-bit, and 16-bit precision, without adding any extra opcode or increasing the complexity of the decode stage. Compared to software-based mixed-precision execution on the baseline *RI5CY* core, such addition provides an improvement of performance and energy efficiency between 1.1× and 4.9×.

[138] introduces a functional unit that implements SIMD MAC with fixed kernel to accelerate CNN inference on the *VexRiscv* soft-core RISC-V processor [163], improving performance by 2.5×.

[56] proposes a multi-precision arithmetic unit to boost the efficiency of the inference of heavily quantized QNNs on microcontroller-class cores by supporting a custom RISC-V ISA extension, *XpulpNN*, including 4-bit and 2-bit packed-SIMD instructions and most notably sum-of dot-product operations that fuse a dot product with a load operation. This results into a 1.64× performance improvement compared to executing standard MAC operations.

The *FPnew* [110] open-source transprecision **floating-point unit (FPU)** implements hardware support for custom packed-SIMD instructions targeting the *float16*, *bfloat16*, and *float8* formats. These additional instructions enable a 1.67× faster execution and a 37% reduction in energy consumption compared to a baseline *float32*-only FPU.

Finally, [86] extends *CVA6* [187] to add hardware support for the RISC-V P extension, limited, however, to the basic and MAC subsets. The obtained core provides 6.7× better performance than the baseline *CVA6* when executing DNN operations while occupying 7.2% more LUTs and 0.6% more FFs at the same 70 MHz clock frequency.

3.1.3 Vector Processors. A large number of solutions have been designed to provide specific support for vector processing in RISC-V architectures, ever since the release of its first specifications and CPU cores [99, 193], and many of the most recent ones are compliant with the RISC-V V extension.

Ara [133] is a RISC-V V 64-bit vector unit, that is, meant to act as a coprocessor for the *CVA6* core and implements a scalable microarchitecture composed of a set of identical lanes, each containing

Table 1. Summary of Main State-of-the-art SIMD Solutions

		Features	Metrics
3.1.2 Packed-SIMD	*RI5CY* [58]	Fixed-point and saturated arithmetic operations; *int8*, *int16*, *int32* formats; Zbb to support BNNs	67 MOPS/mW 65 nm CMOS; 193 MOPS/mW 28 nm FD-SOI
	MPIC [128]	2–16b int formats; targets mixed-precision QNNs	1.1–4.9× energy efficiency vs *RI5CY*
	XpulpNN [56]	2–16b int; fused sum-dot+load instruction	1.64× performance vs standard MAC
	FPnew [110]	Transprecision FP operations; supports *float16*, *bfloat16*, *float8* formats	1.67× speed, −37% energy vs baseline *float32* FPU
3.1.3 Vector Processors	*Ara* [132]	Coprocessor for *CVA6*; scalable vector lanes; full compliance with RISC-V V 1.0	*int8*: 78.4 GOPS, 34.6 GOPS/W; *float64*: 37.8 GFLOPS/W
	Quark [6]	Sub-byte DNNs; strips FP support from *Ara*; adds pop.-count, shift-accumulate, bit-packing	2× less area, 1.9× less power vs *Ara*
	Sparq [48]	Multiply-shift-accumulate for 1–4b convolutions; strips FP support from Ara	−43% area, −59% power, 3.2× (2b), 1.7× (4b) speedup vs *Ara*
	Vecim [178]	SRAM-based IMC vector register file; *int8*, *float16*, *bfloat16* operations	*int8*: 32 GOPS, 289 GOPS/W; *float16*: 25 GFLOPS, 230 GFLOPS/W
	RISC-V^2 [131]	Vector register renaming; decoupled execution; hardware support for reductions	CNN throughput: 1.5× with register renaming, 1.9× with reductions
	[4]	Configurable lanes and length; targets *RI5CY*	2.3× throughput vs base *RI5CY*
3.1.4 Systolic Arrays	*Gemmini* [59]	*bfloat16* 10×10 array; *RoCC*/*Chipyard* integration	127/1760/144× speedup vs *Rocket* (*MobileNetV2*/*SqueezeNet*/*BERT*)
	CHIMERA [139]	16×16 weight-stationary array; 9 MACs/PE, unrolling 3×3 filters	0.92 TOPS; 2.2 TOPS/W
	VSA [109]	Vector unit reused as systolic array; dual modes for CNN layers	Up to 3.5× speedup, −70% memory energy vs baseline vector unit
	[74]	4×4 array w/ *RI5CY* cluster; 2–16b int, *float16* operations	14× throughput vs *RI5CY* with *XpulpNN*
3.1.5 In-Memory Computing	[77]	Charge-domain; *RI5CY*-coupled; runtime-configurable 1–8b	Up to 400 TOPS/W, 0.6 TOPS/mm^2
	AI-PiM [173]	IMC as CPU functional unit; custom MVM and load/store instructions	17.6× MVM speedup, 2.7× *MLPerf Tiny* speedup
	Open-RIMC [130]	RRAM-based; BNN-suitable MACs	XNOR-based: 5.6 TOPS/W; AND-based: 14.0 TOPS/W
	PipeCIM [30]	Pipeline-integrated; SRAM-based; 16 tiles ×128 units, 4b analog MACs	819 GOPS; 134 TOPS/W; 128/19× speedup (*VGG-16*/*Inception*)
	CIMR-V [66]	SRAM-based; custom RISC-V extension for IMC; layer/weight fusion; pipelined max-pooling	26 TOPS @50 MHz; 3708 TOPS/W; −85% latency on **keyword spotting** (**KWS**)

part of the processor's vector **register file (RF)** and functional units. *Ara* exhibits 8-bit integer peak performance of 78.4 GOPS and energy efficiency of 34.64 GOPS/W, while *float16* computing tops at 39.2 GFLOPS and 17.32 GFLOPS/W. Its latest version *Ara2* [132] further improves *float64* energy efficiency up to 37.8 GFLOPS/W, also delivering the first fully open-source vector processor to support the RISC-V V 1.0 frozen ISA.

Quark [6] is an integer RISC-V vector processor for the inference of sub-byte quantized DNNs and based on *Ara* that lowers the resource utilization by removing FP support and instantiates instead hardware support for population-count, shift-and-accumulate, and bit-packing instructions to enable effective bit-serial computation. These changes reduce area and power consumption by 2× and 1.9× compared to *Ara*.

Sparq [48] also strips FP support from *Ara* and adds a custom multiply-shift-accumulate instruction meant for sub-byte computation, targeting convolutions on vectors with 1-bit to 4-bit elements.

Sparq exhibits a reduction in area of 43.3% and in power consumption of 58.8% compared to baseline *Ara*, and a speedup of 3.2× and 1.7×on convolutions with 2-bit and 4-bit quantization, respectively.

Vecim [178] extends *Ara* with IMC capabilities by executing 8-bit integer and 16-bit *bfloat16* and *float16* multiplications and additions in a **static random-access memory (SRAM)**-based vector RF. *Vecim* delivers 8-bit integer peak performance of 31.8 GOPS and energy efficiency of 289.13 GOPS/W, while *float16* computing tops at 25.3 GFLOPS and 230.10 GFLOPS/W. Peak performance is lower than *Ara*'s, but energy efficiency is one order of magnitude higher.

The *RISC-V*^{2} [131] scalable high-performance RISC-V vector processor implements a vector-specific register-renaming mechanism to facilitate dynamic hardware loop unrolling and alleviate instruction dependencies, while a cost-efficient decoupled execution scheme splits instructions into execution and memory-access streams and hardware support for reductions accelerates the execution of key instructions. Register remapping and reduction operations improve the throughput, when executing CNN workloads, by 1.5× and 1.9×, respectively.

[4] integrates in a *RI5CY* [58] core a parametric vector processing unit, configurable in its vector length and number of execution lanes and that implements a subset of RISC-V V, resulting in a throughput improvement of up to 2.3× compared to the baseline processor, while [112] proposes a RISC-V V vector unit that supports precise vector traps and the out-of-order and speculative execution of vector instructions.

Other vector processors include *Arrow* [7], *Spatz* [24], *Vitruvius+* [116], *Klessydra-T13* [26], and *Vicuna* [137]. *Arrow* [7] and *Spatz* [24] are vector processors that implement RISC-V V's integer embedded subset. *Vitruvius+* [116] supports a hybrid in-of-order and out-of-order vector instruction execution scheme by means of register renaming and by decoupling arithmetic and memory instructions. *Klessydra-T13* [26] is a four-stage RV32IMA processor that delivers interleaved-multithreading with a thread context switch at each clock cycle, in-order, single issue instruction execution, a feed-forward pipeline, and support for bare metal execution. The timing-predictable *Vicuna* [137] is meant for real-time applications and features predictable and repeatable timing behavior, thus enabling effective and tight **worst-case execution time (WCET)** analysis. Programmable-precision vector accelerators built through a generator-based design methodology that leverages Chisel [8] are included instead in the RISC-V SoC introduced in [152].

Further optimizations include combining an **exclusive OR (XOR)**-based approach with double pumping to reduce the number of read and write ports in a vector RF [88] and employing an analog IMC array architecture using **resistive RAM (RRAM)** to deliver RISC-V V compliance [192].

3.1.4 Systolic Array Architectures. The open literature includes some RISC-V architectures that make use of systolic arrays, due to the effective mapping of DL-related tasks, in particular matrix multiplications, to their bidimensional structure.

Gemmini [59] delivers an open-source *bfloat16* 10×10 systolic-array accelerator that can be easily integrated with a RISC-V host CPU through its ***Rocket custom co-processor*** (***RoCC***) interface as well as in large heterogeneous multi-core SoC obtained by means of the *Chipyard* framework [5].

The DNN accelerator included in the *CHIMERA* SoC [139] is based on a weight-stationary systolic array of 16×16 **processing elements (PEs)**, each of whom consists of nine parallel MACs, therefore allowing to fully unroll filters up to 3×3. It achieves a peak performance of 0.92 TOPS and a peak energy efficiency of 2.2 TOPS/W.

VSA [109] is a vector-systolic architecture that reuses the components in a vector processing unit to offer the functionality of a systolic array for matrix multiplications with a minimal area overhead below 0.1% compared to a baseline RISC-V implementation with a vector processing unit. Experimental results carried out on *AlexNet*, *ResNet-18* and *ResNet-50* CNN models showcase how the ability to execute the operations of a layer with *VSA* operating as a vector processor or as a

systolic array, depending on which mode provides the best performance, enables a speedup of up to 3.5× and a reduction in memory energy consumption of up to 70%. For instance, systolic-array mode is shown to be the best choice for convolutional layers due to data reuse, while vector-processing mode is shown to perform better on fully-connected ones.

[74] proposes a 4×4 systolic array, that is, tightly coupled with a cluster of *RI5CY* cores and whose 16 PEs support both fixed-point, for inference purposes and with run-time configurable 2-bit to 16-bit integer multipliers, and FP arithmetic, to enable on-device learning and reusing 16-bit integer multipliers for *float16* multiplications. Experimental results encompassing the *MobileNetV2*, *VGG-16*, *ResNet-18*, *ResNet-50*, and *ViT/B-16* DNN models with an 8-bit integer quantization demonstrate 14× higher throughput and energy efficiency on average compared to a *RI5CY* core with support for the *XpulpNN* custom extension[56].

3.1.5 In-Memory Computing Architectures. IMC is an energy-efficient solution to carry out complex computations such as MVMs. RISC-V solutions have therefore emerged that exploit IMC to maximize the efficiency of edge platforms tasked with DL workloads.

[77] integrates in a *RI5CY* core [58], as a tightly coupled accelerator, an analog charge-domain IMC array that enables performing MVMs with input elements having a flexible width that can be configured at run time between 1 and 8 bits. In the experimental evaluation, the IMC array achieves up to 400 TOPS/W and up to 0.6 TOPS/mm^2 on workloads consisting of MVMs with 1-bit elements.

AI-PiM[173] proposes the integration of an IMC array for MVMs, independent from its underlying technology, as a tightly-coupled functional unit for AI in RISC-V CPUs, similarly to how FPUs are commonly integrated in processors to deal with FP-intensive workloads. In addition to a matrix-vector multiplication custom instruction, dedicated load and store ones are also implemented to move output vectors and weights between the IMC array memory and the processor's registers. Experiments show that *AI-PiM* provides speedups of 17.63× and 2.74× on MVM and *MLPerf Tiny* benchmark [10] workloads, respectively, compared to a baseline RV64IMC core.

Open-RIMC [130] is an open-source RRAM-based IMC accelerator for MAC operations, that is, meant to be integrated in a RISC-V processor and, that is, suitable to BNN applications. The experimental evaluation, carried out on a CNN trained over the *MNIST* dataset, shows peak energy efficiency of 5.62 and 14.02 TOPS/W for XNOR-based and AND-based computations, respectively.

PipeCIM [30] integrates, in the pipeline of a RISC-V CPU, a SRAM-based IMC array composed of 16 tiles, each including 128 processing units that perform 4-bit analog MAC computations. Experiments highlight a peak throughput of 819 GOPS, an energy efficiency of 134 TOPS/W, and a speedup of 128× and 19× when executing *VGG-16* and *Inception* CNNs, respectively, compared to the baseline core without IMC capabilities.

CIMR-V [66] is another SRAM-based IMC accelerator integrated with a RISC-V processor, designed to enable end-to-end DL inference through dedicated IMC-type instructions and architectural optimizations such as layer fusion and weight fusion. By integrating a pipelined max-oooling block and reducing feature map data movement, *CIMR-V* achieves 26 TOPS at 50 MHz and an energy efficiency of 3,708 TOPS/W in TSMC 28 nm technology. Experiments on a KWS model demonstrate an 85% latency reduction compared to previous SRAM-based IMC designs.

3.1.6 MIMD Architectures. Some state-of-the-art MIMD solutions implement a hybrid architecture that can be reconfigured at run time to operate as vector processors.

big.VLITTLE [165] architectures are able to reconfigure multiple LITTLE cores of *big.LITTLE* mobile SoCs on demand to work as a decoupled vector engine when executing data-parallel workloads. *big.VLITTLE* is shown to improve performance by 1.6× compared to a big.LITTLE system equipped with an integrated vector unit when executing data-parallel workloads and by 1.7× compared to a vector processor when executing task-parallel workloads.

The *Dustin* [129] 16-core fully programmable compute cluster can either operate as a MIMD architecture or by having a single core fetch instructions and broadcast them to the other 15 cores in a vector processing fashion. In the latter case, switching off instruction fetch stages and private instruction caches of secondary cores saves up to 38% of the whole cluster's power consumption. Moreover, *Dustin* improves performance by 2.7× and energy efficiency by 4.2× compared to the baseline 16-core processor when executing *ResNet-8* inference.

Other RISC-V MIMD architectures can instead be employed at run time as systolic arrays.

The *SNCPU* systolic neural CPU processor [82] enhances the simpler, non-systolic *NCPU* [78] with an architecture that can be dynamically reconfigured as 10 RISC-V CPU cores, a 10×10 systolic MAC array, or a combination of the two. Experiments showcase a limited area overhead of 9.8% compared to the same systolic array without CPU functions, while performance improves by more than 39% compared to *Gemmini* on DNN models for image classification, reaching up to 95% utilization of PEs and an energy efficiency of 1.8 TOPS/W.

[141] proposes an hybrid architecture that enables using a shared-memory many-core RISC-V cluster as a systolic array by implementing specific hardware to support cores communicating with each other through memory-mapped queues that can be accessed through a single instruction, while dedicated queue-linked registers allow managing accesses to the corresponding queues concurrently to the other instructions under execution on the CPU core.

3.2 Data Representation

DL processing commonly operates at lower precision levels than usual computing tasks, due to the remarkable error tolerance exhibited by the majority of DL applications and moreover to the negative impact of excessive precision tracking, that can easily lead to artifacts in the learning process. The selection of the target data representation is a critical choice, since it impacts the ability to employ some parallelisation techniques, such as packed-SIMD processing, and it can dramatically influence the resource utilization and energy consumption required by data processing.

Selecting the number system used to represent real numbers is the primary key decision for what concerns data representation, followed by the selection of the proper data size and precision.

RISC-V hardware architectures for DL at the edge from the open literature can be mainly organized depending on whether they implement fixed-point, FP, or Posit arithmetic. Table 2 summarizes and compares the most notable state-of-the-art solutions.

3.2.1 Fixed-Point Arithmetic. Basic implementation for the I and M RISC-V extensions through a traditional ALU is sufficient to deliver support for fixed-point arithmetic, but dedicated optimized architectures enable further optimizing the performance and efficiency of fixed-point computations in DL-related workloads, also making use of sub-word formats.

[142] designs a bit-serial multiplier for variable-precision integer operands with a width ranging from 1 to 16 bits, meant to enable a custom variable-precision vector extension implemented on a *PicoRV32* RISC-V processor [185]. Compared to the baseline core, throughput on MVMs improves by 14% at the cost of 72% LUT and 16% FF area overheads.

[194] proposes a functional unit for custom RISC-V instructions implementing multiplications and divisions between 32-bit fixed-point operands that produces a minimal 4% area overhead over a multiplication-division unit which supports the standard 32-bit RISC-V M extension. The fixed-point unit is shown to be 1.5× and 2.7× more energy-efficient than with an FPU and with an integer multiplication-division unit, respectively.

[172] integrates in a *RI5CY* core [58] an 18-bit fixed-point approximate multiplier, meant for both simple multiplications and MACs, which is based on the linear **coordinate rotation digital computer (CORDIC)** algorithm and requires only shift-and-add operations. The approximate multiplier consumes 12% less power and has a 27% smaller latency than *RI5CY*'s multiplier.

Table 2. Summary of Main State-of-the-art Solutions Classified According to Data Representation

		Features	Metrics
3.2.1 **Fixed-Point**	[142]	Bit-serial multiplier; variable precision 1–16b integer operands; vector extension for *PicoRV32*	+14% MVM throughput; +72%/16% LUT/FF vs base *PicoRV32*
	[194]	ALU with fixed-point multiplication/division; reuses logic for integer multiplication/division	1.5×/2.7× energy efficiency vs base core with/without *float32* FPU
	[172]	18b approximate multiplier for *RI5CY*; CORDIC-based; shift-and-add operations only	−12% power, −27% latency vs baseline *RI5CY* multiplier
	[125]	NN accelerator with fixed-point or stochastic MAC; bit-serial and bit-parallel modes	Up to 98.6% MNIST accuracy; negligible accuracy loss vs software
3.2.2 **Floating-Point**	*FPUx* [104]	FPU with hybrid pipeline/FSM microarchitecture	−36% energy, 1.3× speedup vs *FPnew*
	[119]	FPUs shared among cores in *RI5CY* clusters; logarithmic-tree interconnect	5.92 GFLOPS; 167 GFLOPS/W; 3.5 GFLOPS/mm^2
	ExSdotp [14]	8/16b custom FP formats; stochastic rounding; fused sum-of-dot-product	1.93× energy efficiency vs *FPnew*
	[195]	Modular FPU; per-operation precision defined at design time; no custom extension	19% energy efficiency improvement over baseline *float32* FPU
	[11]	Adder using HUB FP representation; simplified rounding and computation	−25% area vs baseline FPU adder
	[98]	DNN training accelerator using BFP format; reuses same exponent among multiple FP values	−67% training time, −72% energy vs *bfloat16 Gemmini*
3.2.3 **Posit**	*PERCIVAL* [113]	Extends *CVA6* with 32b Posit, quire operations	1.32× area, 1.38× power overheads vs *FPnew*; higher accuracy vs *float64*
	PPU-light [35]	Format conversions between 8/16b Posit and *float32*/*int32*; integrates with *CVA6*	92/82% 16b, 90/82% 8b Posit accuracy (*LeNet*/*EfficientNet*); 8 mW on FPGA
	Melodica [154]	Parametric quire fused operations	1.5× speedup vs *float32* FPU
	[65]	Scalar and vector Posit arithmetic units	1.61× area reduction, 6.6× energy efficiency vs *Ara*

[125] presents a NN accelerator, that is, configured at design time to implement either a fixed-point variable-precision MAC or stochastic MAC, both of which can operate either in bit-serial or bit-parallel fashion. Stochastic operations notably have a reduced area but require numbers with a larger representation; for instance, stochastic multiplication corresponds to bitwise AND. Experiments show the accelerator's capability to achieve up to 98.55% accuracy on CNNs trained on the *MNIST* dataset, with negligible loss compared to software execution.

[176] proposes a CNN accelerator which leverages an 8-bit dynamic fixed-point representation [84] that shares a scaling factor among a group of numbers, e.g., the weights of a CNN layer. The CNN accelerator, as well as the overall RISC-V-based SoC architecture, are obtained through **high-level synthesis (HLS)** starting from their C++ description.

3.2.2 Floating-Point Arithmetic. RISC-V cores implementing F, D, and V extensions all make use of FP arithmetic instructions, including addition, multiplication, division, square root, multiply-accumulate, and conversions with integer formats and between FP ones.

The open literature includes FPUs that provide support for the standard FP operations and formats included in the RISC-V specification, such as the area-optimized *Tiny-FPU* [16].

The *FPUx* [104] FPU implements a hybrid microarchitecture that combines the pipelining and finite state machine techniques to drive the arithmetic components performing the FP instructions, thus saving clock cycles compared to a traditional pipelined approach. *FPUx* delivers energy savings

of 36% and 1,430% compared to the baseline RISC-V core integrating the *FPNew* FPU and no FPU, respectively, also improving performance on a set of FP-intensive workloads by 1.3× and 17×.

[119] shares one or more FPUs among *RI5CY* cores in a cluster through a logarithmic tree interconnect. In particular, FPUs are statically mapped to cores through a partial interconnect, i.e., each group of cores always accesses the same FPU, with round-robin arbitration to manage simultaneous requests for FP operations. The interconnect interface notably integrates in the execution stage of the CPU cores. Experiments show sixteen-core clusters delivering highest performance of 5.92 GFLOPS with private per-core FPUs with one pipeline stage and highest energy efficiency of 167 GFLOPS/W with zero pipeline stages, while an eight-core cluster with four shared FPUs with one pipeline stage provides highest area-efficiency of 3.5 GFLOPS/mm^2.

Other works implement instead custom operations and formats, often targeting smaller ones to maximize the efficiency in applications such as DL ones that are resilient to approximations.

The *FPnew* [110] transprecision FPU supports multi-format instructions that process inputs and that output results with different FP formats, e.g., multiplications between 16-bit FP values that produce a 32-bit result, while the *ExSdotp* [15] unit provides efficient hardware support for two 8-bit and two 16-bit custom FP formats, expanding operations, fused sum-of-dot-product instructions that accumulate the result in a larger format, and three-term additions. [14] further extends *ExSdotp* with support for the stochastic rounding method. Combining the optimizations in [15] and [14] improves energy efficiency by 1.93× compared to baseline *FPnew*.

[195] proposes a modular FPU whose precision of each operation type can be configured independently at design time, while leveraging the RISC-V F extension and only supporting formats with an 8-bit exponent to avoid the need for compiler changes. The FPU is integrated in a RV32IM CPU core [197].

[11] introduces an adder, employed in a RISC-V vector processor [161], for the **half-unit biased (HUB)** FP representation [73]. This simplifies round-to-nearest rounding and two's complement computations by avoiding carry propagation, without causing neither a precision loss nor larger storage requirement and reducing area by 25% compared to the baseline FPU adder.

The hardware accelerator for DNN training introduced in [98] makes use of a **block-floating-point (BFP)** [180] format, i.e., a variant of the FP representation that reuses the same exponent among multiple values, thus reducing the size of the latter and simplifying some computations. The block size and precision are selected dynamically by configuring, through custom RISC-V instructions, the functional units supporting BFP arithmetic. Experiments demonstrate reductions in CNN training time and energy consumption of 67% and 72%, respectively, compared to employing a *bfloat16 Gemmini* systolic array with similar area.

3.2.3 Posit Arithmetic. The reduction in hardware utilization and the effectiveness for both DL inference [94] and training [108] purposes make Posit arithmetic a promising alternative to traditional IEEE 754 FP representations, thus driving the emergence of a number of RISC-V-based solutions that provide support for it.

PERCIVAL [113] extends *CVA6* with support for all the 32-bit Posit instructions and the quire fused operations of a custom RISC-V extension through dedicated arithmetic unit and RF. The PERCIVAL unit incurs 1.32× area and 1.38× power overheads compared to the *float32*-compliant *FPnew* [110] FPU. 32-bit Posit execution matches the performance of *float32* in GEMM and max-pooling operations from CNN models like *LeNet-5*, *AlexNet*, and *ResNet-50*, while achieving orders-of-magnitude higher numerical accuracy and outperforming *float64* by 1.4-1.7×.

The *PERI* [169] functional unit leverages the standard RISC-V F extension, thus limiting the required modifications to the compiler. It supports design-time configuration of the exponent size and Posit size through a parametric design, while at run time it can instead dynamically switch between two exponent sizes, according to a dedicated RISC-V **control/status register (CSR)**.

The *PPU-light* [35] functional unit implements conversions between 8-bit or 16-bit Posit and 32-bit IEEE 754 FP or fixed-point formats and was integrated into a *CVA6* RISC-V core. *PPU-light* achieves the same accuracy as *float32* execution when using 16-bit Posits, reaching 91.8% and 82.0% for *LeNet* and *EfficientNet*, respectively. With 8-bit Posits, accuracy remains high at 90.4% and 82.1%. Hardware instances supporting these formats demonstrated power consumption of approximately 5 mW for 8-bit and 8 mW for 16-bit Posits when implemented on an *AMD Artix-7 100* FPGA operating at 40 MHz and 50 MHz, respectively [136].

The *Melodica* arithmetic unit [154] implements parametric fused operations on the quire data type. It was demonstrated to achieve up to 1.5× speedup compared to execution using the *float32*-compliant FPU integrated in the same RISC-V core.

Finally, [65] demonstrates Posit's suitability to DL tasks at the edge, implementing scalar and vector Posit arithmetic units and leveraging them to perform channel estimation in multi-input multi-output communication systems. Together with a custom ISA extension supporting operations including matrix inversion and a coarse-grained matrix accelerator, this results into a 1.61× area reduction and an energy efficiency improvement of more than 6.6× compared to *Ara* [133].

3.3 Application-Specific Design

Designing an architecture for DL requires choosing whether the goal is obtaining a more general-purpose computing platform that can execute different workloads or conversely a custom-tailored one, that is, specifically optimized for a narrower use case. The latter option, which includes the adoption of hardware accelerators or custom functional units, favors the performance and energy efficiency metrics at the cost of limited or null usability even in slightly different applications.

By extending the RISC-V ISA with specialized instructions for DL, e.g., MVM, convolution, and activation functions, hardware accelerators can be integrated to enhance performance while reducing latency. Tight coupling, where the accelerator is closely integrated with the processor, e.g., via a shared memory or bus, can offer significant performance gains for specific workloads, but may limit flexibility for other tasks. Loose coupling, where accelerators operate more independently, such as separate processing units or coprocessors, can improve versatility across different applications at the cost of some performance overhead due to communication between processor and accelerator.

The open literature includes RISC-V CPU cores with custom-tailored architectures that enable them to implement a specific convolution operation, a 4-layer BNN, and provide security against side-channel attacks, respectively.

[177] adds dedicated support for a convolution operation with a 3×3 kernel, a 4×4 input matrix, and a 2×2 output matrix by leveraging the Winograd algorithm [181]. This enables computing, with a single custom RISC-V ISA instruction, 16 multiplications rather than the 36 ones otherwise required by the baseline ISA, therefore taking 19 clock cycles instead of 140 for the target convolution.

NCPU [78] is a reconfigurable architecture for low-cost embedded systems that can switch at run time between operating as a five-stage in-order RV32I CPU pipeline and as a neural pipeline implementing a four-layer BNN. Experiments demonstrate 35% area and 12% energy savings over a comparable heterogenous SoC coupling a CPU core with a BNN accelerator, as well as a 43% speedup when executing BNN models for real-time image and motion detection.

[46] proposes an RV32I processor that can execute DNNs securely against SCA attacks by applying masking, leveraging custom-tailored RISC-V instructions, to the execution of NN operations including weighted summations, activation functions, and output layer computations.

Other works from the state of the art deliver instead a number of application-specific hardware accelerators meant for integration with RISC-V cores and that provide support for the various families of DL models, including less common ones such as GCNs, TNNs, and TCNs.

The *CUTIE* [149] accelerator minimizes data movement during TNN inference by keeping all weights on-chip in a compressed format and fully spatially unrolling the multiplications, achieving an energy efficiency of 1,036 TOPS/W. *TCN-CUTIE* [150] extends *CUTIE* to implement a flexible, fully digital accelerator for both TNNs and time-dilated TCNs.

The *E-LSTM* [156] RISC-V-based heterogeneous edge SoC features an LSTM co-processor which exploits a sparse representation for matrices' weights and leverages such sparsity through a cell fusion optimization, increasing the inference throughput between 1.4× and 2.2× on a set of LSTM layers compared to execution on a baseline RISC-V CPU core.

The *multiPULPly* [50] NN inference accelerator implements highly parallel analog multiplications and leverages memristors to consume power only when the memory is active and continue the task with no context-restoring overhead. *multiPULPly* achieves a peak throughput of 82 GOPS when operating at 0.8V and a peak energy efficiency of 19.5 TOPS/W at 0.5V, and delivers a 24× *MobileNet* speedup compared to the baseline *RI5CY* core.

Other notable examples include the multi-precision CNN engine for face recognition in sub-mW operating scenarios introduced in [81], achieving up to 5.76 GOPS throughput and 1.07 TOPS/W energy efficiency; the *XNE* XNOR neural engine [37], a tightly coupled accelerator for BNNs that computes both binarized convolutional and dense layers; the hardware accelerator for generic YOLO-based object detection and classification in [134], that achieves a throughput of 32 and 31 FPS for *YOLOv3-Tiny* and *YOLOv4-Tiny*; and the *GPGCN* [166] accelerator for GCNs, tightly coupled through a *ROCC* interface and that delivers a speedup of more than 1,001× and 267× compared to the baseline *BOOM* core without and with vector processing support, respectively.

Another emerging trend is the use of **application-specific instruction set processors (ASIPs)**, leveraging RISC-V's modular ISA to integrate custom instructions and specialized units for DL at the edge. Commercial design tools such as *Synopsys ASIP Designer*, *Codasip Studio*, and *Andes Technologies*' *COPILOT* framework support the creation of RISC-V-based ASIPs optimized for specialized operations including DL-related ones. Although powerful, these tools are often closed-source and lack extensive public benchmarking, limiting their adoption in open research. Nonetheless, they reflect the growing industrial interest in RISC-V ASIP solutions for edge AI.

3.4 Memory-Side Optimization

As previously discussed, DL workloads are computing-intensive and dominated by MVMs. However, the large amount of data to be processed, as well as the multi-dimensionality of the aforementioned matrices, make it paramount to also carefully design the memory architecture of the computing platform, minimize memory accesses, and optimize the usage of the caches and of the registers inside the processing cores.

Cache memories play a critical role in reducing memory latency and improving data locality. For high-performance systems, using large, multi-level cache hierarchies ensures that frequently used data remains close to the processing units, minimizing the need for slower accesses to main memory. In addition, streaming memory structures, such as dedicated memory banks or on-chip memory, can be particularly beneficial for DL tasks that involve large, continuous data streams, e.g., batch processing of inputs in neural networks. These structures help reduce data movement and increase throughput by providing faster, more efficient access to data. Moreover, fine-tuning memory access patterns and leveraging memory-side computation, e.g., near-memory processing, can further optimize the system by reducing the data transfers between memory and processor.

Some works specifically optimize RISC-V architectures to efficiently deal with memory streams.

[153] proposes **stream semantic registers (SSRs)** to implicitly encode, in custom RISC-V instructions, the memory accesses as register reads/writes by mapping memory streams with

regular address patterns to such SSRs, thus eliminating a large number of load/stores otherwise issued for each ALU or FPU instruction. **Sparse SSRs (SSSRs)** [148] extend SSRs to accelerate the indirection, intersection, and union operations employed in general sparse linear algebra kernels.

Snitch [188] enhances a RISC-V core featuring a double-precision FPU with SSRs and with a micro-loop buffer from which the FP pipeline sequences instructions thus practically decoupling it from the integer pipeline. This significantly reduces the pressure on the core and frees it up for other tasks, making the control core and the FPU effectively dual-issue with a minimal area overhead and without implementing a more complex vector processor architecture.

The performance benefits of these memory-side optimizations are substantial. SSRs have been shown to provide 2×–5× speedups on regular, loop-dominated workloads common in DL inference, primarily by reducing the number of memory instructions and improving cache usage [153]. *Snitch*'s integration of SSRs into a tightly coupled microarchitecture supports effective dual-issue execution with a minimal area overhead of 3.2%, delivering over 6× speedup and 35× energy-efficiency gains on parallel microkernels [188]. SSSRs extend this capability to irregular access patterns typical of sparse DL models. In single-core evaluations, the adoption of SSSRs enables speedups of up to 9.8× and achieves 80% peak FPU utilization in sparse-dense and sparse-sparse operations, while multi-core experiments report up to 5.9× speedups and 3× energy-efficiency improvements over an optimized RISC-V baseline [148]. These results highlight the impact of tightly integrated, stream-oriented memory support in RISC-V architectures for edge-oriented DL workloads.

Other works from the open literature optimize memory accesses by overlapping MACs and loads, implementing a **direct memory access (DMA)** engine that can transfer multi-dimensional data, computing near the **last level cache (LLC)**, and employing non-SRAM memories to store weights in a more energy-efficient way, respectively.

The *Marsellus* [36] end-node heterogeneous SoC includes a 16-core general-purpose cluster that features a hardware functional unit for fused MAC-and-load packed-SIMD instructions, achieving up to 94% utilization of MAC units in compute-intensive kernels rather than a baseline 27%.

The *iDMA* [12] DMA engine can manage transfers with more than one addressing dimension, i.e., transfers of two-dimensional, three-dimensional, and generic multi-dimensional tensors, which are common in DL workloads, and provides a front-end that can be tightly coupled to a RISC-V core thus enabling the encoding of DMA transfers as custom RISC-V instructions, such as in *Snitch* [188]. The adoption of *iDMA* in a multi-core edge node improves *MobileNet* inference performance from 7.9 to 8.3 MACs per cycle compared to a traditional DMA engine, while reducing area occupation by 10%.

[27] presents an eight-core 64-bit RISC-V processor that performs MAC operations within the shared LLC by adding a MAC unit to the LLC datapath and therefore performing their computation near the SRAM arrays that constitute the LLC. Cache memories do not require any modification to implement the proposed approach.

The *CHIMERA* SoC [139] couples a RISC-V CPU and a DNN accelerator with on-chip RRAM to store all the weights, thus minimizing the energy consumption due to memory accesses compared to traditional **dynamic RAM (DRAM)**, while the *Siracusa* SoC [140] features a 4 MiB on-chip **spin-transfer torque magnetic RAM (STT-MRAM)** and employs it, due to its high density, high bandwidth, and non-volatile nature, to store the DNN weights to be fed to a DNN inference accelerator, thus eliminating off-chip weight transfers.

4 Software Tools

Software support to optimize computing platforms for DL at the edge can be provided through a variety of different solutions. At a coarse grain, we can differentiate between tools that offer co-design capabilities, thus enabling the design of the hardware platform together with the software

optimization, and tools that focus on software optimization for a given hardware platform. In addition, the latter can be partitioned into autotuning frameworks, which modify just the parameters of the software, and compilers, that actually modify the code.

Software support for DL on RISC-V computing platforms at the edge has been recently ramping up thanks to several contributions both in research and in industrial development. While many research projects started out primarily focusing on hardware architectures and provided instead only minimal software support for code generation geared towards some specific extension, the emergence of *MLIR* [95], or *Multi-Level Intermediate Representation*, within the compiler construction community is nowadays providing a common environment to develop such extensions.

This section, after providing an overview of the main software frameworks for RISC-V edge DL deployment, highlights the primary efforts towards co-designing RISC-V-based hardware and software as well as tuning the performance and optimizing the code, classifying the software tools from the open literature that support RISC-V edge computing platforms for DL.

4.1 Software Frameworks for RISC-V Edge Deep-Learning Deployment

Software frameworks for DL have recently started targeting the deployment of workloads on RISC-V-based platforms at the edge.

TensorFlow Lite for Microcontrollers provides a lightweight runtime tailored to deeply embedded systems, supporting quantized models and ensuring compatibility with RISC-V cores through its modular backend [168].

TVM offers a flexible compilation framework that enables hardware-specific optimizations via autotuning and tensorization [29]. Its integration with *LLVM* and support for custom code generation allow *TVM* to target RISC-V architectures, including those with packed-SIMD and vector extensions [31], while recent efforts have further extended it to exploit ISA extensions for DL acceleration [184].

A key limitation of these frameworks lies in their standalone nature, which prevents them from leveraging the broader set of optimization passes developed within the open-source compiler community. To mitigate this issue, some frameworks have started to adopt the *MLIR* infrastructure as a back-end, from which RISC-V code can be generated and further extended with DL-specific ISA features through dedicated external dialects.

4.2 Hardware-Software Co-Design

HW/SW co-design is a well-known technique that enables the generation of application specific systems by tailoring the hardware and the software at the same time, thus exploiting synergies and removing unnecessary hardware components. In the context of RISC-V, where the baseline architecture is defined, we can speak more properly of platform-based design, which, within the spectrum of co-design techniques, aims at simplifying the design and development process by adopting a common platform, that is, tailored through the co-design techniques. Specifically, two classes of approaches are found in the literature, namely *ISA extension* and *multiprecision* support.

Recent explorations [68] have demonstrated that the interplay between algorithmic and micro-architectural optimization is significant and, as a result, HW/SW co-design can offer substantial benefits in terms of performance, area, and energy efficiency. However, HW/SW co-design also necessitates a considerable amount of tooling, thus limiting the number of available efforts, most of which are therefore very recent. Front-end and optimization frameworks such as *TVM* [29] and *TensorFlow Lite for Microcontrollers* [168] are commonly employed, together with various solutions meant to generate code for specialized platforms.

Considering specifically the exploitation of sparsity, recent works [52, 144] propose HW/SW co-design methodologies for accelerating sparse DNNs on FPGAs using RISC-V extensions, such as

RISC-V V. These works combine dedicated hardware features with software-level management of sparsity to achieve denser, and therefore faster, computations.

For a recent overview of HW/SW co-design tools, both general-purpose and specific to DL and RISC-V, which is beyond the scope of this article, we refer the reader to [122, 189].

4.2.1 ISA Extension and Code-Generator Generation. ISA extensions can be co-designed in conjunction with software, facilitating a more precise alignment between the ISA and the software's requirements. This approach minimizes the risk of certain extensions being underutilized in practical applications. Nonetheless, this methodology necessitates dual support from the co-design tools. On the one hand, the tools must be capable of generating the ISA extension and its microarchitectural implementation. On the other hand, the compiler backend's code generator must be automatically extended to support the newly introduced instructions.

EXTREM-EDGE [174] aims at providing a full HW/SW co-design approach, including both the generation of RISC-V ISA extensions dedicated to accelerating AI operations, and the software support to convert neural networks to the assembly code for the generated platform. The software components leverage *TVM* [29] to ingest and optimize neural networks in a variety of languages. The C back-end of *TVM* is then exploited to feed the generated C code to *Synopsys ASIP Designer*, which generates an SDK able to translate from C to the extended RISC-V. While this choice provides a robust solution which leverages two well-known tools, the use of the C language as an **intermediate representation (IR)** is likely to limit the optimization opportunities, since domain specific information is lost when using a general purpose source language as an intermediate compiler representation [18].

The *Scale4Edge* project [49] develops a similar platform-based design concept, where ISA extensions are introduced in a RISC-V-based ecosystem that supports primarily C programming, but also supports *TensorFlow Lite for Microcontrollers* as well as *TVM* for the deployment of DL models. The focus of *Scale4Edge* is on edge computing, which explains the choice of TinyML.

Within the framework of *Scale4Edge*, [20] describes three approaches to design space exploration for a HW/SW co-design of edge AI systems, including one for systems with tightly-coupled and loosely-coupled accelerator architectures based on RISC-V. In particular, the authors propose the *HANNAH* design flow to automatically co-optimize neural network architectures and a corresponding neural network accelerator. In the proposed flow, NNs are first instantiated and trained in the training component of *HANNAH* employing quantization-aware training and dataset augmentation. Trained NNs are then handed over to the deployment component for target code generation, where the NN is quantized to a low word width representation, the NN operations are scheduled, and on-device memory is allocated for the neural network. Along with the target NN architecture, a specialized NN hardware accelerator is instantiated from a configurable Verilog template. Hardware-dependent performance metrics like power consumption and chip area are then either generated by running the NN on a gate-level simulation or estimated using an analytical performance model. The evolution-based search strategy implemented in the *HANNAH* optimizer is then used to incrementally search the NN and hardware accelerator co-design space. *HANNAH* is meant to be integrated with the *TVM* compiler, while the accelerator is loosely coupled with a RISC-V processor. The authors apply the proposed methodology to the *UltraTrail* [13] ultra-low-power *TC-ResNet* AI accelerator for KWS, consisting of a configurable array of PEs and a distributed memory with dynamic content reallocation.

[9] explores custom extension of RISC-V ISA with three new instructions which combine arithmetic operations and memory load. While the approach provides a full toolchain based on PyTorch, the majority of the effort is focused on selecting and implementing the new instructions, while providing support for them at the *LLVM* level.

LiteAIR5 [55] aims at providing a full framework for processor and memory customization, employing the AIR5 RISC-V core as the basic platform to which additional AI-specific functional units can be added by a HW/SW co-design process. However, compiler support is still fairly simplistic, with the need to explicitly write assembly code for the new instructions which is manually integrated in the main application code through the C inline assembly feature.

The *CoreDLS* language and its companion HLS tool *Longnail* [127] address the specific issue of ISA extension, aiming at making it easier to manually specify the extension with a high level language syntax, and then compile it to hardware modules that can be integrated employing the *SCAIE-V* vendor-agnostic hardware interface generator [41].

[67] explores co-design opportunities for the RISC-V V extension, focusing on exploring the parameters of CNNs and architectural features, in particular, vector length and cache memory size.

4.2.2 Multiprecision. Multiprecision is another HW/SW co-design technique that provides instead support for multiple levels of precision in the supported number systems at both the hardware and software levels. This technique similarly requires support at the compiler level, and specific ISA extensions may also be utilized. However, the scope of these extensions is more constrained, therefore simplifying the implementation of autotuning tools.

[71] proposes an agile HW/SW co-design to generate a RISC-V-based multi-precision DL microprocessor. The authors designed custom RISC-V instructions as well as a framework that complements *TVM* to automatically compile multi-precision CNN networks onto multi-precision CNN accelerators. The evaluation on *AMD* FPGAs demonstrated that the cores generated from the proposed methodology improved over the throughput of state-of-the-art solutions with both 2-bit, 4-bit, and 8-bit precisions and NAS-optimized multi-precision CNN models such as *LeNet*, *VGG16*, *ResNet*, and *MobileNet*.

Another solution, proposed in [43], enables the tuning of precision between fixed-point and FP at the compiler level, and the selection of an optimal configuration of the FPU depending on the workload profile. This solution targets a RISC-V soft core with a customizable FPU [195] and employs a co-design approach leveraging compile-time application profiling.

At a smaller scale, [191] proposes a multiprecision RISC-V processor supporting small integer arithmetic at precisions from 2 to 16 bits, and then explores mixed-precision quantization strategies that take advantage of the architectural flexibility.

4.3 Autotuning Frameworks

Autotuning is a recognized methodology for enhancing the performance of complex software systems through the automatic adjustment of their parameters. This process encompasses strategies such as alternating between various compiled versions of the same software, modifying compilation parameters, or adjusting a set of parameters made accessible by the application. Crucially, autotuning employs a black-box approach to optimization, relying on the manipulation of available options that are exposed as configurable knobs rather than altering the software's underlying code.

PULP-TrainLib [123] is a software library for DNN training on RISC-V-based multi-core MCUs that includes a variety of latency-optimized and performance-tunable forward and backward training primitives of DNN layers. On-device DNN training is further enabled by coupling *PULP-TrainLib* with the *AutoTuner* tool to automatically identify the fastest configuration and tile partitioning for each DNN layer. Experimental evaluation of the proposed flow is carried out targeting an 8-core RISC-V-based PULP platform.

[184] extends the compiler flow developed in [32] (see Section 4.4.1) with an autotuning mechanism to control the radix point position for fixed-point types employed.

4.4 Compiler Technology

Compiler support for RISC-V, encompassing both frameworks and specific optimizations, can be categorized into three main areas.

The first one consists of *DL compilers*, which transform models described by frameworks such as *TensorFlow*, *PyTorch*, or *ONNX* into lower-level representations, analogously to how traditional compilers convert source code written in high-level programming languages, such as C or C++, into machine code executable by general-purpose hardware. The produced lower-level code can be made more efficient by tailoring it to the specific characteristics of DL workloads, ensuring optimal execution on various hardware backends, including CPUs and specialized accelerators.

The second category involves the compiler extensions, i.e., custom modifications to the compiler, that are necessary to support specialized RISC-V ISA extensions and in particular recognize a set of idioms and generate the corresponding instructions from the ISA extension.

Finally, the third one pertains instead to compiler transformations, which include loop optimizations, memory optimizations, and precision tuning among the most commonly employed in the open literature.

4.4.1 DL Compiler Frameworks. *IREE* [105] is a unified compiler and run-time stack with the goal to scale down ML to the smallest footprints for mobile and edge devices while maintaining the ability to scale up to larger deployment targets. *IREE* adopts a compiler-based approach and optimizes for heterogeneous hardware accelerators through the use of the *MLIR* compiler infrastructure. *TinyIREE* is a set of deployment options in *IREE* that accommodate the limited memory and computation resources in embedded systems and bare-metal platforms. Support is provided for the RISC-V ISA.

The *CuPBoP* framework [70] executes *CUDA* programs on non-*Nvidia* devices without relying on other programming languages. The compilation part of *CuPBoP* applies transformations on *CUDA* host/kernel IRs, while its run-time part consists of the run-time libraries for *CUDA* built-in functions. Backend support for the RISC-V ISA is notably included, enabling the execution of *CUDA* programs on RISC-V CPUs.

[32] studies the optimization of the code generation process for a RISC-V core supporting the P extension, starting from the *TVM* back-end of *LLVM* and appropriately configuring *TVM* to employ its tensorization step to generate *LLVM* intrinsics that will be then lowered to the RISC-V P's packed-SIMD instructions. While this flow is effective, it highlights the limitations of *LLVM* in dealing with custom semantics, which must universally be represented with opaque intrinsics. This, in turn, can limit the optimization capabilities of the compiler. A further extension presented in [186] provides support for custom instructions as well.

4.4.2 Loop Optimization. Autovectorization and slicing and tiling transformations are the most common loop optimizations. Autovectorization facilitates the use of packed subword operations or vector accelerators and it is essential for the adoption of the latter, since manually writing vectorized code is cumbersome and not typically supported by general-purpose programming languages. Slicing and tiling transformations enhance instead matrix operations by reordering the execution sequence, such as in block matrix multiplication, to optimize memory locality and thereby reduce memory bandwidth pressure.

[162] proposes the *TSO* (tensor slicing optimization) compiler pass for multi-core **neural processing units (NPUs)**, meant to be integrated in a flow using the *Tensorflow XLA* domain-specific compiler for linear algebra and *LLVM*. *TSO* aims to maximize convolution parallelism and memory usage across NPU cores and reduce data transfers between host and NPU on-chip memories by using DRAM memory burst time estimates to guide tensor slicing. The experimental evaluation targets the *LG Electronics' NMP* neuromorphic processor, a multi-core NPU containing 32 RISC-V

cores extended with novel CNN instructions, showing that *TSO* can identify the best tensor slicing that minimizes execution time for a set of CNN models.

[1] evaluates the compiler shortcomings that constitute the gap in autovectorization capabilities between length-agnostic and fixed-length architectures. The authors examine *LLVM*'s support for the RISC-V V extension, studying a set of synthetic scalar loops to compare the compiler support in the length-agnostic and fixed-length cases and then examining a benchmark suite to compare autovectorized to hand-vectorized RISC-V code.

Finally, [107] pioneers the usage of *MLIR* and streaming registers, aiming at high utilization of the computational resources.

4.4.3 Memory Optimization. Specialized memory architectures, such as streaming processors, and specialized DMA controllers, such as those reported in Section 3.4, require explicit management at compiler level, and general-purpose code must be adapted to suit the unique characteristics of these architectures.

The *HEROv2* compiler [92] provides an *AutoDMA* plugin that performs an analysis on the intermediate representation of an OpenMP program targeting the *HEROv2* heterogeneous platform endowed with RISC-V accelerators, identifying suitable memory regions and partitioning the kernels in load, execute and store phases, and then mapping the first and last to DMA API calls. The experimental assessment highlights a 4.4× reduction of the execution time.

[153] proposes a compiler transformation to exploit their SSRs, a hardware construct that extends the RISC-V ISA to implicitly encode memory accesses as register reads and writes. The proposed transformation is applied to constant-bound loop nests, and detects all constant stride memory accesses, marking them as candidates for mapping to the SSRs. It then employs a simple heuristic based on the nesting depth to prioritize the memory accesses, and allocates them in order to the SSRs. Finally, configuration instructions are emitted to set up the SSRs. Some additional fine tuning is needed to avoid transforming small loops, which would lead to a significant setup overhead with respect to the savings provided by the stream registers. A theoretical and experimental analysis shows that the proposed technique provides speedups in the range of 2× to 5×.

[124] introduces a compiler analysis to automatically identify memory access patterns to generate efficient code targeting a stream-based execution model. Since stream-based execution leverages implicit memory accesses, which are not modeled by the Static Single Assignment semantics of *LLVM*, the authors specialize the compilation flow to generate the stream model assembly by detecting the streams in the *LLVM* IR, and encoding them with what amounts to a specialized representation. It is worth noting that such an approach suffers from a fundamental weakness in that it performs the detection of relatively high-level semantics on the low-level *LLVM* IR. A more modern approach would leverage *MLIR* to directly detect the stream semantics in the high-level code, encode them in a dedicated dialect, and then perform code generation towards a specialized external dialect, directly convertible to the target assembly. While the efficiency gains of this higher-level approach have not yet been evaluated, the higher-level semantics of *MLIR* make, at least, the analysis and transformation easier to maintain [95].

4.4.4 Precision Tuning. Precision tuning is a code transformation technique that adjusts the precision of computations, potentially enabling further optimizations such as vectorization through a sufficient reduction of the data size. This technique utilizes low-precision arithmetic operations to balance accuracy with memory bandwidth, thanks to data size reduction, and resource utilization, e.g., by switching from requiring an FPU to using a baseline ALU, and it can be viewed as a generalization of the quantization process in DL applications.

Precision tuning is indeed a key feature for DL frameworks, since quantization is one of their main steps. As such, most frameworks described in Section 4.4.1 provide some support for precision tuning. This subsection focuses specifically on dedicated tools.

[154] presents a *GCC*-based toolchain for the RISC-V ISA to support the Posit number system, including support for programs mixing IEEE 754 FP and Posit representations, particularly focusing on high-precision computing. Experimental results are provided for the execution of linear algebra and computer vision kernels on a custom RISC-V core (*Clarinet*) synthesized on an *AMD* FPGA.

The *TAFFO* precision-tuning framework [34], which is implemented as a plugin for the *LLVM* compiler framework and operates on both C and C++ sources, allows converting selected FP arithmetic operations to fixed-point ones, depending on performance and accuracy requirements set by the designer. It was notably employed on a RISC-V CPU implementing hardware support for both FP and fixed-point arithmetic in [194]. Furthermore, the interactions of precision tuning with the automated management of DMA described in Section 4.4.3 have been explored in [111].

5 Summary and Future Directions

This manuscript surveyed the existing research related to RISC-V computing platforms at the edge for DL workloads, delivering a structured taxonomy of the solutions from the open literature by splitting them between hardware architectures and software tools and frameworks.

In particular, the RISC-V hardware architectures for DL at the edge are classified according to how they process data in parallel, how they represent data, whether they implement an application-specific design, and how they optimize the crucial aspect of data movement, while software tools supporting such RISC-V architectures are organized into HW/SW co-design, autotuning, and compiler frameworks and optimizations.

5.1 Key Findings

Our review of the state of the art has highlighted six key findings.

(1) Multitude of open-hardware cores. The open-standard nature of RISC-V, coupled with its flexible royalty-free license that allows derivative works to be either open-source or closed-source and free or proprietary, has spurred the development and the public release of a large number of RISC-V cores. The available open-hardware solutions range from designs specialized for DL applications [37, 44, 59, 130] to more general-purpose scalar ones [58, 100, 187], the latter of which can be possibly aided by dedicated software libraries to deliver effective DL processing capabilities at the edge [40, 123].

(2) Widespread implementation of hardware for vector processing. A vast amount of RISC-V hardware solutions for SIMD processing is available, in particular with support for the variable-length vector V and packed-SIMD P extensions. However, while a large number of architectures for RISC-V vector processing supports the standard RISC-V V extension [24, 116, 133, 137, 152], the set of available packed-SIMD computing solutions capable of supporting DL applications is more fragmented, since few implement RISC-V P [86, 101] and a majority of them rely instead on custom instructions [56, 58, 110, 138]. Notably, the specification of the official RISC-V P packed-SIMD extension is still being defined and during its development has shifted more towards DSP applications than DL ones.

(3) Limited capabilities for compiler-level optimization. From the point of view of compiler technology, the current landscape is supporting DL on RISC-V architectures is dominated by the use of toolchains such as *TVM* [29] and *Tensorflow* [168], which employ either internal code generators, usually to produce LLVM IR, or a source-to-source approach, by generating equivalent C code

which is then translated to assembly by an external compiler. This solution, while effective, can be cumbersome, since LLVM does not support directly specialized instructions. Instead, the toolchain developer must rely on opaque intrinsics, which may hamper optimization of the rest of the code. Opaque intrinsics are indeed characterized by having their semantics unknown to the compiler, thus compiler passes must consider them as boundaries for the region under optimization, which means having, on average, smaller regions and possibly giving up optimization opportunities provided by instructions on different sides of the boundary.

(4) Mature fixed-point and FP hardware and software support. A huge fraction of works, both on the hardware and software sides, targets the traditional fixed-point and IEEE 754 FP number systems. Support at the hardware level, including fixed-point packed-SIMD processing [58] and computations on run-time configurable formats [194] as well as lightweight [16] and transprecision [104, 110, 119] FPUs, is effectively complemented on the software side by precision tuning tools [58], autotuning frameworks [123], and optimizations carried out at DL compiler level [105].

(5) Resilience, predictability, and security in safety-critical DL at the edge. The deployment of DL at the edge in safety-critical domains requires hardware and software platforms that combine fault resilience, predictable real-time performance, and robust security. Recent advances in the RISC-V ecosystem, including lockstep [146] and hybrid modular redundancy [60, 143], reconfigurable fault-tolerant architectures [38], and timing-predictable vector coprocessors [85, 137], are being complemented by root-of-trust platforms such as *lowRISC*'s *OpenTitan* and countermeasures against side-channel attacks [46, 197] to ensure trustworthy execution of DL workloads under stringent certification standards. These trends highlight safety, predictability, and trustworthiness as key enablers for RISC-V adoption in regulated, safety-critical edge AI applications such as automotive and space [53, 170].

(6) Sparsity-aware hardware and software for energy efficiency. Sparsity of DL models offers significant opportunities to reduce the memory and computational cost of their inference at the edge. Solutions such as *E-LSTM* [156] exploit sparsity in recurrent models via custom co-processors, while SSRs and their SSSRs [148, 153] provide hardware support for optimized memory streaming suited to sparse linear algebra operations. Architectures like *Snitch* [188] combine these mechanisms with micro-loop buffers to decouple pipelines, substantially improving performance and energy efficiency on sparse workloads. Recent advances in HW/SW co-design of RISC-V extensions further enable flexible and efficient acceleration of semi-structured and unstructured sparse DNNs on FPGAs [52, 144, 156], while domain-specific accelerators integrated into SoCs with RISC-V cores demonstrate practical sparse DL inference [91]. Collectively, these advances underscore the importance of sparsity-aware HW/SW support in meeting the stringent performance and energy requirements of future systems for DL at the edge.

5.2 Future Directions

Our comprehensive investigation of the open literature related to RISC-V computing platforms for DL at the edge has led us to identify five main promising directions for future research.

(1) Alternative data representations. A limited number of state-of-the-art solutions explored the usage of alternative data representations, and the few ones are limited to the adoption of Posit arithmetic in RISC-V architectures [35, 113, 169]. For example, the open literature does not include any hardware support [57] for the **logarithmic number system (LNS)** and its approximate, reduced-precision variants [3] on RISC-V architectures. We envision future research efforts that explore, in particular through HW/SW co-design approaches, new data formats such as LNS for which hardware support can deliver improvement in latency, accuracy, area, or energy efficiency

for some specific DL applications, including training, which is nowadays still performed for the most part in centralized premises rather than at the edge.

(2) Novel IMC technologies for highly parallel DL processing. IMC has emerged as an effective solution, in particular from the point of view of energy efficiency, for processing MVMs, which are by far the most common operations in DL workloads. However, its usage in actual RISC-V computing platforms at the edge is still limited nowadays [30, 65, 77]. This limited adoption can be attributed to several factors, including technological barriers such as the immaturity of current IMC hardware solutions and challenges in integrating emerging technologies like STT-MRAM [135] and **ferroelectric field-effect transistors (FeFETs)** [93, 160] into practical systems. In addition, the existing software ecosystem lacks adequate support for IMC, as most toolchains and compilers are not equipped to handle its unconventional computation models. Enabling widespread adoption will therefore require not only hardware innovation but also dedicated compiler support and new programming models that can be effectively mapped onto RISC-V architectures.

(3) Extension of MLIR ecosystem to RISC-V. An alternative, emerging solution on the compiler side is provided by the *MLIR* framework [95], a multi-level intermediate representation where *LLVM* IR can co-exist with higher level representations, and optimizations can be implemented for specific dialects. The *ONNX-MLIR* project [79] aims at generating MLIR for neural networks represented using the *ONNX* interchange format, while *Torch-MLIR* [106] and *TensorFlow-MLIR* [167] provide MLIR backends for the respective toolchains. The MLIR ecosystem, which is rapidly developing as the common grounds between the DL toolchains, has also seen an increase in support for machine-specific dialects, including support for GPUs from both *Nvidia* and *AMD*, as well as *Arm* SIMD extensions. The missing link remains the development of dedicated MLIR dialects for RISC-V extensions, which can be foreseen as a step that will take place in the next few years. A strong MLIR-based ecosystem coupling DL frameworks and RISC-V extensions will be crucial to the implementation of more ambitious toolchains, e.g., aiming at platform-based co-design.

(4) Dedicated hardware and software support for emerging DL models. On the one hand, the exponential growth in size of DL models, which has reached hundreds of billions in **large language models (LLMs)** [25], makes it even more critical to optimize the data movement within the system in computing platforms at the edge, which requires acting both at the hardware and software level. Further research will therefore have to be devoted to identifying even more capable and energy-efficient solutions than the latest state-of-the-art ones that effectively support multi-dimensional transfers through DMA engines [12], memory streams also of sparsely represented data [148], and quantization [111]. On the other hand, we envision the definition of novel RISC-V ISA extensions for instructions that are common in DL workloads, such as MVMs, convolutions, or graph operations employed in emerging GCN models, together with their dedicated support ranging from DL compiler backends to compiler optimizations and hardware functional units.

(5) Dedicated support for on-device training and federated learning. Despite promising experimental demonstrations of on-device training and federated continual learning on RISC-V edge platforms, several key challenges remain before these technologies can be broadly practical and effective [98, 118, 123]. Current limitations include the lack of efficient hardware primitives tailored for training workloads under ultra-low power and memory constraints, as well as the absence of standardized software frameworks that can robustly orchestrate federated learning across heterogeneous and dynamic edge networks [90]. Moreover, hardware support for secure aggregation, communication efficiency, and privacy-preserving computations is still immature or missing from many RISC-V implementations [17]. Future research should prioritize HW/SW co-design of lightweight federated learning middleware optimized for RISC-V extensions, integration of quantized and mixed-precision

training algorithms, and upstream compiler and ML library support to enable seamless exploitation of emerging hardware features. Addressing these gaps will be crucial to unlock scalable, reliable, and privacy-aware on-device learning capabilities in decentralized edge environments.

References

[1] Neil Adit and Adrian Sampson. 2022. Performance left on the table: An evaluation of compiler autovectorization for RISC-V. *IEEE Micro* 42, 5 (2022), 41–48. DOI: 10.1109/MM.2022.3184867

[2] Ghattas Akkad, Ali Mansour, and Elie Inaty. 2024. Embedded deep learning accelerators: A survey on recent advances. *IEEE Transactions on Artificial Intelligence* 5, 5 (2024), 1954–1972. DOI: 10.1109/TAI.2023.3311776

[3] Syed Asad Alam, James Garland, and David Gregg. 2021. Low-precision logarithmic number systems: Beyond base-2. *ACM Transactions on Architecture and Code Optimization* 18, 4, Article 47 (Jul 2021), 25 pages. DOI: 10.1145/3461699

[4] Muhammad Ali, Matthias von Ameln, and Diana Goehringer. 2021. Vector processing unit: A RISC-V based SIMD co-processor for embedded processing. In *Proceedings of the 2021 24th Euromicro Conference on Digital System Design (DSD)*. 30–34. DOI: 10.1109/DSD53832.2021.00014

[5] Alon Amid, David Biancolin, Abraham Gonzalez, Daniel Grubb, Sagar Karandikar, Harrison Liew, Albert Magyar, Howard Mao, Albert Ou, Nathan Pemberton, et al. 2020. Chipyard: Integrated design, simulation, and implementation framework for custom SoCs. *IEEE Micro* 40, 4 (Jul 2020), 10–21. DOI: 10.1109/MM.2020.2996616

[6] MohammadHossein AskariHemmat, Théo Dupuis, Yoan Fournier, Nizar El Zarif, Matheus Cavalcante, Matteo Perotti, Frank Gürkaynak, Luca Benini, François Leduc-Primeau, Yvon Savaria, and Jean-Pierre David. 2023. Quark: An integer RISC-V vector processor for sub-byte quantized DNN inference. In *Proceedings of the 2023 IEEE International Symposium on Circuits and Systems (ISCAS)*. 1–5. DOI: 10.1109/ISCAS46773.2023.10181985

[7] Imad Al Assir, Mohamad El Iskandarani, Hadi Rayan Al Sandid, and Mazen A. R. Saghir. 2021. Arrow: A RISC-V Vector Accelerator for Machine Learning Inference. 1–6. arXiv:2107.07169. Retrieved from https://arxiv.org/abs/2107.07169

[8] Jonathan Bachrach, Huy Vo, Brian Richards, Yunsup Lee, Andrew Waterman, Rimas Avižienis, John Wawrzynek, and Krste Asanović. 2012. Chisel: Constructing hardware in a scala embedded language. In *Proceedings of the 49th Annual Design Automation Conference* (San Francisco, California) *(DAC'12)*. Association for Computing Machinery, New York, NY, USA, 1216–1225. DOI: 10.1145/2228360.2228584

[9] Karthikeyan Kalyanasundaram Balasubramanian, Mirco Di Salvo, Walter Rocchia, Sergio Decherchi, and Marco Crepaldi. 2024. Designing RISC-V instruction set extensions for artificial neural networks: An LLVM compiler-driven perspective. *IEEE Access* 12 (2024), 55925–55944. DOI: 10.1109/ACCESS.2024.3389673

[10] Colby Banbury, Vijay Janapa Reddi, Peter Torelli, Nat Jeffries, Csaba Kiraly, Jeremy Holleman, Pietro Montino, David Kanter, Pete Warden, Danilo Pau, et al. 2021. MLPerf tiny benchmark. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, J. Vanschoren and S. Yeung (Eds.), Vol. 1. Retrieved from https://datasets-benchmarks-proceedings.neurips.cc/paper_files/paper/2021/file/da4fb5c6e93e74d3df8527599fa62642-Paper-round1.pdf

[11] Gerardo Bandera, Javier Salamero, Miquel Moreto, and Julio Villalba. 2024. Floating Point HUB Adder for RISC-V Sargantana Processor. 1–2. arXiv:2401.09464. Retrieved from https://arxiv.org/abs/2401.09464

[12] Thomas Benz, Michael Rogenmoser, Paul Scheffler, Samuel Riedel, Alessandro Ottaviano, Andreas Kurth, Torsten Hoefler, and Luca Benini. 2024. A high-performance, energy-efficient modular DMA engine architecture. *IEEE Transactions on Computers* 73, 1 (2024), 263–277. DOI: 10.1109/TC.2023.3329930

[13] Paul Palomero Bernardo, Christoph Gerum, Adrian Frischknecht, Konstantin Lübeck, and Oliver Bringmann. 2020. UltraTrail: A configurable ultralow-power TC-resnet AI accelerator for efficient keyword spotting. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 39, 11 (2020), 4240–4251. DOI: 10.1109/TCAD.2020.3012320

[14] Luca Bertaccini, Gianna Paulin, Matheus Cavalcante, Tim Fischer, Stefan Mach, and Luca Benini. 2024. MiniFloats on RISC-V cores: ISA extensions with mixed-precision short dot products. *IEEE Transactions on Emerging Topics in Computing* 12, 4 (2024), 1040–1055. DOI: 10.1109/TETC.2024.3365354

[15] Luca Bertaccini, Gianna Paulin, Tim Fischer, Stefan Mach, and Luca Benini. 2022. MiniFloat-NN and ExSdotp: An ISA extension and a modular open hardware unit for low-precision training on RISC-V cores. In *Proceedings of the 2022 IEEE 29th Symposium on Computer Arithmetic (ARITH)*. 1–8. DOI: 10.1109/ARITH54963.2022.00010

[16] Luca Bertaccini, Matteo Perotti, Stefan Mach, Pasquale Davide Schiavone, Florian Zaruba, and Luca Benini. 2021. Tiny-FPU: Low-cost floating-point support for small RISC-V MCU cores. In *Proceedings of the 2021 IEEE International Symposium on Circuits and Systems (ISCAS)*. 1–5. DOI: 10.1109/ISCAS51556.2021.9401149

[17] Sizhen Bian, Pixi Kang, Julian Moosmann, Mengxi Liu, Pietro Bonazzi, Roman Rosipal, and Michele Magno. 2024. On-device learning of EEGNet-based network for wearable motor imagery brain-computer interface. In *Proceedings of the 2024 ACM International Symposium on Wearable Computers* (Melbourne VIC, Australia) *(ISWC'24)*. Association for Computing Machinery, New York, NY, USA, 9–16. DOI: 10.1145/3675095.3676607

[18] João Bispo, Nuno Paulino, and Luís Miguel Sousa. 2023. Challenges and opportunities in C/C++ source-to-source compilation. In *Proceedings of the 14th Workshop on Parallel Programming and Run-Time Management Techniques for Many-Core Architectures and 12th Workshop on Design Tools and Architectures for Multicore Embedded Computing Platforms (PARMA-DITAM 2023)*. Schloss Dagstuhl-Leibniz-Zentrum für Informatik.

[19] Sérgio Branco, André G. Ferreira, and Jorge Cabral. 2019. Machine learning in resource-scarce embedded systems, FPGAs, and end-devices: A survey. *Electronics* 8, 11 (2019). DOI: 10.3390/electronics8111289

[20] Oliver Bringmann, Wolfgang Ecker, Ingo Feldner, Adrian Frischknecht, Christoph Gerum, Timo Hämäläinen, Muhammad Abdullah Hanif, Michael J. Klaiber, Daniel Mueller-Gritschneder, Paul Palomero Bernardo, Sebastian Prebeck, and Muhammad Shafique. 2021. Automated HW/SW co-design for edge AI: State, challenges and steps ahead. In *Proceedings of the 2021 International Conference on Hardware/Software Codesign and System Synthesis* (Virtual Event) *(CODES/ISSS'21)*. Association for Computing Machinery, New York, NY, USA, 11–20. DOI: 10.1145/3478684.3479261

[21] Vincent Camus, Linyan Mei, Christian Enz, and Marian Verhelst. 2019. Review and benchmarking of precision-scalable multiply-accumulate unit architectures for embedded neural-network processing. *IEEE Journal on Emerging and Selected Topics in Circuits and Systems* 9, 4 (2019), 697–711. DOI: 10.1109/JETCAS.2019.2950386

[22] Luigi Capogrosso, Federico Cunico, Dong Seon Cheng, Franco Fummi, and Marco Cristani. 2024. A machine learning-oriented survey on tiny machine learning. *IEEE Access* 12 (2024), 23406–23426. DOI: 10.1109/ACCESS.2024.3365349

[23] Maurizio Capra, Beatrice Bussolino, Alberto Marchisio, Guido Masera, Maurizio Martina, and Muhammad Shafique. 2020. Hardware and software optimizations for accelerating deep neural networks: Survey of current trends, challenges, and the road ahead. *IEEE Access* 8 (2020), 225134–225180. DOI: 10.1109/ACCESS.2020.3039858

[24] Matheus Cavalcante, Domenic Wüthrich, Matteo Perotti, Samuel Riedel, and Luca Benini. 2022. Spatz: A compact vector processing unit for high-performance and energy-efficient shared-l1 clusters. In *Proceedings of the 41st IEEE/ACM International Conference on Computer-Aided Design* (San Diego, California) *(ICCAD'22)*. Association for Computing Machinery, New York, NY, USA, Article 22, 9 pages. DOI: 10.1145/3508352.3549367

[25] Yupeng Chang, Xu Wang, Jindong Wang, Yuan Wu, Linyi Yang, Kaijie Zhu, Hao Chen, Xiaoyuan Yi, Cunxiang Wang, Yidong Wang, Wei Ye, Yue Zhang, Yi Chang, Philip S. Yu, Qiang Yang, and Xing Xie. 2024. A survey on evaluation of large language models. *ACM Transactions on Intelligent Systems and Technology* 15, 3, Article 39 (mar 2024), 45 pages. DOI: 10.1145/3641289

[26] Abdallah Cheikh, Stefano Sordillo, Antonio Mastrandrea, Francesco Menichelli, Giuseppe Scotti, and Mauro Olivieri. 2021. Klessydra-T: Designing vector coprocessors for multithreaded edge-computing cores. *IEEE Micro* 41, 2 (2021), 64–71. DOI: 10.1109/MM.2021.3050962

[27] Gregory K. Chen, Phil C. Knag, Carlos Tokunaga, and Ram K. Krishnamurthy. 2023. An eight-core RISC-V processor with compute near last level cache in intel 4 CMOS. *IEEE Journal of Solid-State Circuits* 58, 4 (2023), 1117–1128. DOI: 10.1109/JSSC.2022.3228765

[28] Jiasi Chen and Xukan Ran. 2019. Deep learning with edge computing: A review. *Proceedings of the IEEE* 107, 8 (2019), 1655–1674. DOI: 10.1109/JPROC.2019.2921977

[29] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Haichen Shen, Meghan Cowan, Leyuan Wang, Yuwei Hu, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. 2018. {TVM}: An automated {end-to-end} optimizing compiler for deep learning. In *Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. 578–594.

[30] Tingran Chen, Wenjia Wang, Jiaqi Chen, Haotian Fu, Wente Yi, Bojun Cheng, He Zhang, and Biao Pan. 2024. PipeCIM: A high-throughput computing-in-memory microprocessor with nested pipeline and RISC-V extended instructions. *IEEE Transactions on Circuits and Systems I: Regular Papers* 71, 7 (2024), 3214–3227. DOI: 10.1109/TCSI.2024.3384271

[31] Yi-Ru Chen, Hui-Hsin Liao, Chia-Hsuan Chang, Che-Chia Lin, Chao-Lin Lee, Yuan-Ming Chang, Chun-Chieh Yang, and Jenq-Kuen Lee. 2020. Experiments and optimizations for TVM on RISC-V architectures with P extension. In *Proceedings of the 2020 International Symposium on VLSI Design, Automation and Test (VLSI-DAT)*. 1–4. DOI: 10.1109/VLSI-DAT49148.2020.9196477

[32] Yi-Ru Chen, Hui-Hsin Liao, Chia-Hsuan Chang, Che-Chia Lin, Chao-Lin Lee, Yuan-Ming Chang, Chun-Chieh Yang, and Jenq-Kuen Lee. 2020. Experiments and optimizations for TVM on RISC-V architectures with P extension. In *Proceedings of the 2020 International Symposium on VLSI Design, Automation and Test (VLSI-DAT)*. 1–4. DOI: 10.1109/VLSI-DAT49148.2020.9196477

[33] Yu Cheng, Duo Wang, Pan Zhou, and Tao Zhang. 2018. Model compression and acceleration for deep neural networks: The principles, progress, and challenges. *IEEE Signal Processing Magazine* 35, 1 (2018), 126–136. DOI: 10.1109/MSP.2017.2765695

[34] Stefano Cherubin, Daniele Cattaneo, Michele Chiari, Antonio Di Bello, and Giovanni Agosta. 2020. TAFFO: Tuning assistant for floating to fixed point optimization. *IEEE Embedded Systems Letters* 12, 1 (2020), 5–8. DOI: 10.1109/LES.2019.2913774

[35] Marco Cococcioni, Federico Rossi, Emanuele Ruffaldi, and Sergio Saponara. 2022. A lightweight posit processing unit for RISC-V processors in deep neural network applications. *IEEE Transactions on Emerging Topics in Computing* 10, 4 (2022), 1898–1908. DOI: 10.1109/TETC.2021.3120538

[36] Francesco Conti, Davide Rossi, Gianna Paulin, Anaelo Garofalo, Alfio Di Mauro, Georg Rutishauer, Gian marco Ottavi, Manuel Eggimann, Hayate Okuhara, Vincent Huard, et al. 2023. 22.1 A 12.4TOPS/W @ 136GOPS AI-IoT System-on-Chip with 16 RISC-V, 2-to-8b Precision-Scalable DNN Acceleration and 30%-Boost Adaptive Body Biasing. In *Proceedings of the 2023 IEEE International Solid- State Circuits Conference (ISSCC)*. 21–23. DOI: 10.1109/ISSCC42615.2023.10067643

[37] Francesco Conti, Pasquale Davide Schiavone, and Luca Benini. 2018. XNOR neural engine: A hardware accelerator IP for 21.6-fJ/op binary neural network inference. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 37, 11 (2018), 2940–2951. DOI: 10.1109/TCAD.2018.2857019

[38] Giorgio Cora, Eleonora Vacca, Corrado De Sio, Sarah Azimi, and Luca Sterpone. 2025. RePAIR: Reconfigurable platform for AI resilience within RISC-V ecosystem. In *Proceedings of the Applied Reconfigurable Computing. Architectures, Tools, and Applications*, Roberto Giorgi, Mirjana Stojilović, Dirk Stroobandt, Piedad Brox Jiménez, and Ángel Barriga Barros (Eds.). Springer Nature Switzerland, Cham, 71–87.

[39] Enfang Cui, Tianzheng Li, and Qian Wei. 2023. RISC-V instruction set architecture extensions: A survey. *IEEE Access* 11 (2023), 24696–24711. DOI: 10.1109/ACCESS.2023.3246491

[40] Francesco Daghero, Chen Xie, Daniele Jahier Pagliari, Alessio Burrello, Marco Castellano, Luca Gandolfi, Andrea Calimera, Enrico Macii, and Massimo Poncino. 2021. Ultra-compact binary neural networks for human activity recognition on RISC-V processors. In *Proceedings of the 18th ACM International Conference on Computing Frontiers* (Virtual Event, Italy) *(CF'21)*. Association for Computing Machinery, New York, NY, USA, 3–11. DOI: 10.1145/3457388.3458656

[41] Mihaela Damian, Julian Oppermann, Christoph Spang, and Andreas Koch. 2022. SCAIE-V: An open-source SCAlable interface for ISA extensions for RISC-V processors. In *Proceedings of the 59th ACM/IEEE Design Automation Conference* (San Francisco, California) *(DAC'22)*. Association for Computing Machinery, New York, NY, USA, 169–174. DOI: 10.1145/3489517.3530432

[42] Lei Deng, Guoqi Li, Song Han, Luping Shi, and Yuan Xie. 2020. Model compression and hardware acceleration for neural networks: A comprehensive survey. *Proceedings of the IEEE* 108, 4 (2020), 485–532. DOI: 10.1109/JPROC.2020.2976475

[43] Lev Denisov, Andrea Galimberti, Daniele Cattaneo, Giovanni Agosta, and Davide Zoni. 2024. Design-time methodology for optimizing mixed-precision CPU architectures on FPGA. *Journal of Systems Architecture* 155 (2024), 103257. DOI: 10.1016/j.sysarc.2024.103257

[44] A. Di Mauro, M. Scherer, D. Rossi, and L. Benini. 2022. Kraken: A direct event/frame-based multi-sensor fusion soc for ultra-efficient visual processing in nano-UAVs. In *Proceedings of the 2022 IEEE Hot Chips 34 Symposium (HCS)*. IEEE Computer Society, Los Alamitos, CA, USA, 1–19. DOI: 10.1109/HCS55958.2022.9895621

[45] Alexander Dörflinger, Mark Albers, Benedikt Kleinbeck, Yejun Guan, Harald Michalik, Raphael Klink, Christopher Blochwitz, Anouar Nechi, and Mladen Berekovic. 2021. A comparative survey of open-source application-class RISC-V processor implementations. In *Proceedings of the 18th ACM International Conference on Computing Frontiers* (Virtual Event, Italy) *(CF'21)*. Association for Computing Machinery, New York, NY, USA, 12–20. DOI: 10.1145/3457388.3458657

[46] Anuj Dubey, Rosario Cammarota, Avinash Varna, Raghavan Kumar, and Aydin Aysu. 2023. Hardware-software co-design for side-channel protected neural network inference. In *Proceedings of the 2023 IEEE International Symposium on Hardware Oriented Security and Trust (HOST)*. 155–166. DOI: 10.1109/HOST55118.2023.10133716

[47] R. Duncan. 1990. A survey of parallel computer architectures. *Computer* 23, 2 (1990), 5–16. DOI: 10.1109/2.44900

[48] Théo Dupuis, Yoan Fournier, MohammadHossein AskariHemmat, Nizar El Zarif, François Leduc-Primeau, Jean Pierre David, and Yvon Savaria. 2023. Sparq: A custom RISC-V vector processor for efficient sub-byte quantized inference. In *Proceedings of the 2023 21st IEEE Interregional NEWCAS Conference (NEWCAS)*. 1–5. DOI: 10.1109/NEWCAS57931.2023.10198172

[49] Wolfgang Ecker, Peer Adelt, Wolfgang Mueller, Reinhold Heckmann, Milos Krstic, Vladimir Herdt, Rolf Drechsler, Gerhard Angst, Ralf Wimmer, Andreas Mauderer, et al. 2022. The Scale4Edge RISC-V ecosystem. In *Proceedings of the 2022 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. 808–813. DOI: 10.23919/DATE54114.2022.9774593

[50] Adi Eliahu, Ronny Ronen, Pierre-Emmanuel Gaillardon, and Shahar Kvatinsky. 2021. MultiPULPly: A multiplication engine for accelerating neural networks on ultra-low-power architectures. *ACM Journal on Emerging Technologies in Computing Systems* 17, 2, Article 24 (apr 2021), 27 pages. DOI: 10.1145/3432815

[51] Islam Elsadek and Eslam Yahya Tawfik. 2021. RISC-V resource-constrained cores: A survey and energy comparison. In *Proceedings of the 2021 19th IEEE International New Circuits and Systems Conference (NEWCAS)*. 1–5. DOI: 10.1109/NEWCAS50681.2021.9462781

[52] Chao Fang, Wei Sun, Aojun Zhou, and Zhongfeng Wang. 2023. Unleashing the potential of sparse DNNs through synergistic hardware-sparsity co-design. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 42, 4 (2023), 1147–1160. DOI: 10.1109/TCAD.2022.3198756

[53] William Fornaciari, Federico Reghenzani, Giovanni Agosta, Davide Zoni, Andrea Galimberti, Francesco Conti, Yvan Tortorella, Emanuele Parisi, Francesco Barchi, Andrea Bartolini, et al. 2023. RISC-V processor technologies for aerospace applications in the ISOLDE project. In *Proceedings of the Embedded Computer Systems: Architectures, Modeling, and Simulation*, Cristina Silvano, Christian Pilato, and Marc Reichenbach (Eds.). Springer Nature Switzerland, Cham, 363–378.

[54] N. Gala, A. Menon, R. Bodduna, G. S. Madhusudan, and V. Kamakoti. 2016. SHAKTI processors: An open-source hardware initiative. In *Proceedings of the 2016 29th International Conference on VLSI Design and 2016 15th International Conference on Embedded Systems (VLSID)*. IEEE Computer Society, Los Alamitos, CA, USA, 7–8. DOI: 10.1109/VLSID.2016.130

[55] Yimin Gao, Sergiu Mosanu, Mohammad Nazmus Sakib, Vaibhav Verma, Xinfei Guo, and Mircea Stan. 2023. LiteAIR5: A system-level framework for the design and modeling of AI-extended RISC-V cores. In *Proceedings of the 2023 IEEE 36th International System-on-Chip Conference (SOCC)*. 1–6. DOI: 10.1109/SOCC58585.2023.10257058

[56] Angelo Garofalo, Giuseppe Tagliavini, Francesco Conti, Luca Benini, and Davide Rossi. 2021. XpulpNN: Enabling energy efficient and flexible inference of quantized neural networks on RISC-V based IoT end nodes. *IEEE Transactions on Emerging Topics in Computing* 9, 3 (2021), 1489–1505. DOI: 10.1109/TETC.2021.3072337

[57] Michael Gautschi, Michael Schaffner, Frank K. Gürkaynak, and Luca Benini. 2017. An extended shared logarithmic unit for nonlinear function kernel acceleration in a 65-nm CMOS multicore cluster. *IEEE Journal of Solid-State Circuits* 52, 1 (2017), 98–112. DOI: 10.1109/JSSC.2016.2626272

[58] Michael Gautschi, Pasquale Davide Schiavone, Andreas Traber, Igor Loi, Antonio Pullini, Davide Rossi, Eric Flamand, Frank K. Gürkaynak, and Luca Benini. 2017. Near-threshold RISC-V core with DSP extensions for scalable IoT endpoint devices. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 25, 10 (2017), 2700–2713. DOI: 10.1109/TVLSI.2017.2654506

[59] Hasan Genc, Seah Kim, Alon Amid, Ameer Haj-Ali, Vighnesh Iyer, Pranav Prakash, Jerry Zhao, Daniel Grubb, Harrison Liew, Howard Mao, Albert Ou, Colin Schmidt, Samuel Steffl, John Wright, Ion Stoica, Jonathan Ragan-Kelley, Krste Asanovic, Borivoje Nikolic, and Yakun Sophia Shao. 2021. Gemmini: Enabling systematic deep-learning architecture evaluation via full-stack integration. In *Proceedings of the 2021 58th ACM/IEEE Design Automation Conference (DAC)*. 769–774. DOI: 10.1109/DAC18074.2021.9586216

[60] Luigi Ghionda, Riccardo Tedeschi, Yvan Tortorella, Arpan Suravi Prasad, Davide Rossi, Luca Benini, and Francesco Conti. 2025. HMR-NEureka: Hybrid modular redundancy DNN acceleration in heterogeneous RISC-V SoCs. In *Proceedings of the 2025 IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*, Vol. 1. 1–6. DOI: 10.1109/ISVLSI65124.2025.11130209

[61] Garrett B. Goh, Nathan O. Hodas, and Abhinav Vishnu. 2017. Deep learning for computational chemistry. *Journal of Computational Chemistry* 38, 16 (2017), 1291–1307. DOI: 10.1002/jcc.24764

[62] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. 2016. *Deep Learning*. MIT Press.

[63] Samuel Greengard. 2020. Will RISC-V revolutionize computing? *Communications of the ACM* 63, 5 (apr 2020), 30–32. DOI: 10.1145/3386377

[64] Kaiyuan Guo, Shulin Zeng, Jincheng Yu, Yu Wang, and Huazhong Yang. 2019. [DL] A survey of FPGA-based neural network inference accelerators. *ACM Transactions on Reconfigurable Technology and Systems* 12, 1, Article 2 (mar 2019), 26 pages. DOI: 10.1145/3289185

[65] Meng Guo, Qi Wu, Chuanning Wang, Yangcan Zhou, Shaowei Wang, Chuan Zhang, Zhongfeng Wang, and Jun Lin. 2025. A RISC-V domain-specific processor for deep learning-based channel estimation. *IEEE Transactions on Circuits and Systems I: Regular Papers* 72, 5 (2025), 2380–2393. DOI: 10.1109/TCSI.2025.3547319

[66] Yan-Cheng Guo, Tian-Sheuan Chang, Chih-Sheng Lin, Bo-Cheng Chiou, Chih-Ming Lai, Shyh-Shyuan Sheu, Wei-Chung Lo, and Shih-Chieh Chang. 2024. CIMR-V: An end-to-end SRAM-based CIM accelerator with RISC-V for AI edge device. In *Proceedings of the 2024 IEEE International Symposium on Circuits and Systems (ISCAS)*. 1–5. DOI: 10.1109/ISCAS58744.2024.10558177

[67] Sonia Rani Gupta, Nikela Papadopoulou, Jing Chen, and Miquel Pericàs. 2024. Co-design of convolutional algorithms and long vector RISC-V processors for efficient CNN model serving. In *Proceedings of the 53rd International Conference on Parallel Processing* (Gotland, Sweden) *(ICPP'24)*. Association for Computing Machinery, New York, NY, USA, 73–83. DOI: 10.1145/3673038.3673121

[68] Sonia Rani Gupta, Nikela Papadopoulou, and Miquel Pericàs. 2023. Accelerating CNN inference on long vector architectures via co-design. In *Proceedings of the 2023 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. 145–155. DOI: 10.1109/IPDPS54959.2023.00024

[69] John L Gustafson. 2017. *The end of Error: Unum Computing*. Chapman and Hall/CRC.

[70] Ruobing Han, Jun Chen, Bhanu Garg, Jeffrey Young, Jaewoong Sim, and Hyesoon Kim. 2023. CuPBoP: A framework to make CUDA portable. In *Proceedings of the 28th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming* (Montreal, QC, Canada) *(PPoPP'23)*. Association for Computing Machinery, New York, NY, USA, 444–446. DOI: 10.1145/3572848.3577504

[71] Zicheng He, Ao Shen, Qiufeng Li, Quan Cheng, and Hao Yu. 2023. Agile hardware and software co-design for RISC-V-based multi-precision deep learning microprocessor. In *Proceedings of the 28th Asia and South Pacific Design Automation Conference* (Tokyo, Japan) *(ASPDAC'23)*. Association for Computing Machinery, New York, NY, USA, 490–495. DOI: 10.1145/3566097.3567871

[72] Sepp Hochreiter and Jürgen Schmidhuber. 1997. Long short-term memory. *Neural Computation* 9, 8 (1997), 1735–1780. DOI: 10.1162/neco.1997.9.8.1735

[73] Javier Hormigo and Julio Villalba. 2016. New formats for computing with real-numbers under round-to-nearest. *IEEE Transactions on Computers* 65, 7 (2016), 2158–2168. DOI: 10.1109/TC.2015.2479623

[74] Longwei Huang, Chao Fang, Qiong Li, Jun Lin, and Zhongfeng Wang. 2024. A precision-scalable RISC-V DNN processor with on-device learning capability at the extreme edge. In *Proceedings of the 2024 29th Asia and South Pacific Design Automation Conference (ASP-DAC)*. 927–932. DOI: 10.1109/ASP-DAC58780.2024.10473817

[75] Itay Hubara, Matthieu Courbariaux, Daniel Soudry, Ran El-Yaniv, and Yoshua Bengio. 2016. Binarized neural networks. In *Proceedings of the 30th International Conference on Neural Information Processing Systems* (Barcelona, Spain) *(NIPS'16)*. Curran Associates Inc., Red Hook, NY, USA, 4114–4122.

[76] Roland Höller, Dominic Haselberger, Dominik Ballek, Peter Rössler, Markus Krapfenbauer, and Martin Linauer. 2019. Open-source RISC-V processor IP cores for FPGAs — overview and evaluation. In *Proceedings of the 2019 8th Mediterranean Conference on Embedded Computing (MECO)*. 1–6. DOI: 10.1109/MECO.2019.8760205

[77] Hongyang Jia, Hossein Valavi, Yinqi Tang, Jintao Zhang, and Naveen Verma. 2020. A programmable heterogeneous microprocessor based on bit-scalable in-memory computing. *IEEE Journal of Solid-State Circuits* 55, 9 (2020), 2609–2621. DOI: 10.1109/JSSC.2020.2987714

[78] Tianyu Jia, Yuhao Ju, Russ Joseph, and Jie Gu. 2020. NCPU: An embedded neural CPU architecture on resource-constrained low power devices for real-time end-to-end performance. In *Proceedings of the 2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 1097–1109. DOI: 10.1109/MICRO50266.2020.00091

[79] Tian Jin, Gheorghe-Teodor Bercea, Tung D. Le, Tong Chen, Gong Su, Haruki Imai, Yasushi Negishi, Anh Leu, Kevin O'Brien, Kiyokuni Kawachiya, and Alexandre E. Eichenberger. 2020. Compiling ONNX Neural Network Models Using MLIR. 1–8. arXiv:2008.08272. Retrieved from https://arxiv.org/abs/2008.08272

[80] Petar Jokic, Erfan Azarkhish, Andrea Bonetti, Marc Pons, Stephane Emery, and Luca Benini. 2022. A construction kit for efficient low power neural network accelerator designs. *ACM Transactions on Embedded Computing Systems* 21, 5, Article 56 (oct 2022), 36 pages. DOI: 10.1145/3520127

[81] Petar Jokic, Erfan Azarkhish, Regis Cattenoz, Engin Türetken, Luca Benini, and Stephane Emery. 2021. A Sub-mW dual-engine ML inference system-on-chip for complete end-to-end face-analysis at the edge. In *Proceedings of the 2021 Symposium on VLSI Circuits*. 1–2. DOI: 10.23919/VLSICircuits52068.2021.9492401

[82] Yuhao Ju and Jie Gu. 2023. A systolic neural CPU processor combining deep learning and general-purpose computing with enhanced data locality and end-to-end performance. *IEEE Journal of Solid-State Circuits* 58, 1 (2023), 216–226. DOI: 10.1109/JSSC.2022.3214170

[83] Stavros Kalapothas, Manolis Galetakis, Georgios Flamis, Fotis Plessas, and Paris Kitsos. 2023. A survey on RISC-V-based machine learning ecosystem. *Information* 14, 2 (2023). DOI: 10.3390/info14020064

[84] Yongshin Kang and Jaeyong Chung. 2017. A dynamic fixed-point representation for neuromorphic computing systems. In *Proceedings of the 2017 International SoC Design Conference (ISOCC)*. 44–45. DOI: 10.1109/ISOCC.2017.8368819

[85] Maximilian Kirschner, Konstantin Dudzikk, and Jürgen Becker. 2024. Work-in-progress: Real-time neural network inference on a custom RISC-V multicore vector processor. In *Proceedings of the 2024 IEEE Real-Time Systems Symposium (RTSS)*. 427–430. DOI: 10.1109/RTSS62706.2024.00043

[86] Davy Koene. 2021. Implementation and evaluation of packed-SIMD instructions for a RISC-V processor. Retrieved from https://repository.tudelft.nl/islandora/object/uuid%3Ac4162ff8-9419-4434-852d-c1c3297df808

[87] Bastian Koppelmann, Peer Adelt, Wolfgang Mueller, and Christoph Scheytt. 2019. RISC-V extensions for bit manipulation instructions. In *Proceedings of the 2019 29th International Symposium on Power and Timing Modeling, Optimization and Simulation (PATMOS)*. 41–48. DOI: 10.1109/PATMOS.2019.8862170

[88] Nikola Kovačević, Đorđe Mišeljić, and Aleksa Stojković. 2022. RISC-V vector processor for acceleration of machine learning algorithms. In *Proceedings of the 2022 30th Telecommunications Forum (TELFOR)*. 1–4. DOI: 10.1109/TELFOR56187.2022.9983779

[89] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. 2012. ImageNet classification with deep convolutional neural networks. In *Proceedings of the Advances in Neural Information Processing Systems*, F. Pereira, C.J. Burges, L. Bottou, and K.Q. Weinberger (Eds.), Vol. 25. Curran Associates, Inc. Retrieved from https://proceedings.neurips.cc/paper_files/paper/2012/file/c399862d3b9d6b76c8436e924a68c45b-Paper.pdf

[90] Lars Kröger, Cristian Cioflan, Victor Kartsch, and Luca Benini. 2025. On-Device Federated Continual Learning on RISC-V-based Ultra-Low-Power SoC for Intelligent Nano-Drone Swarms. 1–2. arXiv:2503.17436. Retrieved from https://arxiv.org/abs/2503.17436

[91] Vineet Kumar, Ajay Kumar M, Yike Li, Shreejith Shanker, and Deepu John. 2025. Bare-Metal RISC-V + NVDLA SoC for Efficient Deep Learning Inference. arXiv:2508.16095. Retrieved from https://arxiv.org/abs/2508.16095

[92] Andreas Kurth, Björn Forsberg, and Luca Benini. 2022. HEROv2: Full-stack open-source research platform for heterogeneous computing. *IEEE Transactions on Parallel and Distributed Systems* 33, 12 (2022), 4368–4382. DOI: 10.1109/TPDS.2022.3189390

[93] Nellie Laleni, Franz Müller, Gonzalo Cuñarro, Thomas Kämpfe, and Taekwang Jang. 2024. A high-efficiency charge-domain compute-in-memory 1F1C macro using 2-bit FeFET cells for DNN processing. *IEEE Journal on Exploratory Solid-State Computational Devices and Circuits* 10 (2024), 153–160. DOI: 10.1109/JXCDC.2024.3495612

[94] Hamed F. Langroudi, Zachariah Carmichael, John L. Gustafson, and Dhireesha Kudithipudi. 2019. PositNN framework: Tapered precision deep learning inference for the edge. In *Proceedings of the 2019 IEEE Space Computing Conference (SCC)*. 53–59. DOI: 10.1109/SpaceComp.2019.00011

[95] Chris Lattner, Mehdi Amini, Uday Bondhugula, Albert Cohen, Andy Davis, Jacques Pienaar, River Riddle, Tatiana Shpeisman, Nicolas Vasilache, and Oleksandr Zinenko. 2021. MLIR: Scaling compiler infrastructure for domain specific computation. In *Proceedings of the 2021 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. IEEE, 2–14.

[96] C. Lea, M. D. Flynn, R. Vidal, A. Reiter, and G. D. Hager. 2017. Temporal convolutional networks for action segmentation and detection. In *Proceedings of the 2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE Computer Society, Los Alamitos, CA, USA, 1003–1012. DOI: 10.1109/CVPR.2017.113

[97] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner. 1998. Gradient-based learning applied to document recognition. *Proceedings of the IEEE* 86, 11 (1998), 2278–2324. DOI: 10.1109/5.726791

[98] Seunghyun Lee, Jeik Choi, Seockhwan Noh, Jahyun Koo, and Jaeha Kung. 2023. DBPS: Dynamic block size and precision scaling for efficient DNN training supported by RISC-V ISA extensions. In *Proceedings of the 2023 60th ACM/IEEE Design Automation Conference (DAC)*. 1–6. DOI: 10.1109/DAC56929.2023.10248013

[99] Yunsup Lee, Andrew Waterman, Rimas Avizienis, Henry Cook, Chen Sun, Vladimir Stojanović, and Krste Asanović. 2014. A 45nm 1.3GHz 16.7 double-precision GFLOPS/W RISC-V processor with vector accelerators. In *Proceedings of the ESSCIRC 2014 - 40th European Solid State Circuits Conference (ESSCIRC)*. 199–202. DOI: 10.1109/ESSCIRC.2014.6942056

[100] Yunsup Lee, Andrew Waterman, Henry Cook, Brian Zimmer, Ben Keller, Alberto Puggelli, Jaehwa Kwak, Ruzica Jevtic, Stevo Bailey, Milovan Blagojevic, et al. 2016. An agile approach to building RISC-V microprocessors. *IEEE Micro* 36, 2 (2016), 8–20. DOI: 10.1109/MM.2016.11

[101] Kai Li, Wei Yin, and Qiang Liu. 2023. A portable DSP coprocessor design using RISC-V packed-SIMD instructions. In *Proceedings of the 2023 IEEE International Symposium on Circuits and Systems (ISCAS)*. 1–5. DOI: 10.1109/ISCAS46773.2023.10181681

[102] Mingzhen Li, Yi Liu, Xiaoyan Liu, Qingxiao Sun, Xin You, Hailong Yang, Zhongzhi Luan, Lin Gan, Guangwen Yang, and Depei Qian. 2021. The deep learning compiler: A comprehensive survey. *IEEE Transactions on Parallel and Distributed Systems* 32, 3 (2021), 708–727. DOI: 10.1109/TPDS.2020.3030548

[103] Siyuan Liang, Hao Wu, Li Zhen, Qiaozhi Hua, Sahil Garg, Georges Kaddoum, Mohammad Mehedi Hassan, and Keping Yu. 2022. Edge YOLO: Real-time intelligent object detection system based on edge-cloud cooperation in autonomous vehicles. *IEEE Transactions on Intelligent Transportation Systems* 23, 12 (2022), 25345–25360. DOI: 10.1109/TITS.2022.3158253

[104] Xian Lin, Heming Liu, Xin Zheng, Huaien Gao, Shuting Cai, and Xiaoming Xiong. 2024. FPUx: High-performance floating-point support for cost-constrained RISC-V cores. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 32, 10 (2024), 1945–1949. DOI: 10.1109/TVLSI.2024.3399221

[105] Hsin-I Cindy Liu, Marius Brehler, Mahesh Ravishankar, Nicolas Vasilache, Ben Vanik, and Stella Laurenzo. 2022. TinyIREE: An ML execution environment for embedded systems from compilation to deployment. *IEEE Micro* 42, 5 (2022), 9–16. DOI: 10.1109/MM.2022.3178068

[106] LLVM. 2020. The Torch-MLIR Project. Retrieved from https://github.com/llvm/torch-mlir

[107] Alexandre Lopoukhine, Federico Ficarelli, Christos Vasiladiotis, Anton Lydike, Josse Van Delm, Alban Dutilleul, Luca Benini, Marian Verhelst, and Tobias Grosser. 2025. A multi-level compiler backend for accelerated micro-kernels targeting RISC-V ISA extensions *(CGO'25)*. Association for Computing Machinery, New York, NY, USA, 163–178. DOI: 10.1145/3696443.3708952

[108] Jinming Lu, Chao Fang, Mingyang Xu, Jun Lin, and Zhongfeng Wang. 2021. Evaluations on deep neural networks training using posit number system. *IEEE Transactions on Computers* 70, 2 (2021), 174–187. DOI: 10.1109/TC.2020.2985971

[109] Mateo Vázquez Maceiras, Muhammad Waqar Azhar, and Pedro Trancoso. 2022. VSA: A hybrid vector-systolic architecture. In *Proceedings of the 2022 IEEE 40th International Conference on Computer Design (ICCD)*. 368–376. DOI: 10.1109/ICCD56317.2022.00061

[110] Stefan Mach, Fabian Schuiki, Florian Zaruba, and Luca Benini. 2021. FPnew: An open-source multiformat floating-point unit architecture for energy-proportional transprecision computing. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 29, 4 (2021), 774–787. DOI: 10.1109/TVLSI.2020.3044752

[111] Gabriele Magnani, Daniele Cattaneo, Lev Denisov, Giuseppe Tagliavini, Giovanni Agosta, and Stefano Cherubin. 2024. SeTHet – sending tuned numbers via DMA onto heterogeneous clusters: An automated precision tuning story. In *Proceedings of the ACM Computing Frontiers*.

[112] Vincenzo Maisto and Alessandro Cilardo. 2022. A pluggable vector unit for RISC-V vector extension. In *Proceedings of the 2022 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. 1143–1148. DOI: 10.23919/DATE54114.2022.9774501

[113] David Mallasén, Raul Murillo, Alberto A. Del Barrio, Guillermo Botella, Luis Piñuel, and Manuel Prieto-Matias. 2022. PERCIVAL: Open-source posit RISC-V core with quire capability. *IEEE Transactions on Emerging Topics in Computing* 10, 3 (2022), 1241–1252. DOI: 10.1109/TETC.2022.3187199

[114] Danilo Mandic and Jonathon Chambers. 2001. *Recurrent Neural Networks for Prediction: Learning Algorithms, Architectures and Stability*. Wiley.

[115] Benjamin W. Mezger, Douglas A. Santos, Luigi Dilillo, Cesar A. Zeferino, and Douglas R. Melo. 2022. A survey of the RISC-V architecture software support. *IEEE Access* 10 (2022), 51394–51411. DOI: 10.1109/ACCESS.2022.3174125

[116] Francesco Minervini, Oscar Palomar, Osman Unsal, Enrico Reggiani, Josue Quiroga, Joan Marimon, Carlos Rojas, Roger Figueras, Abraham Ruiz, Alberto Gonzalez, et al. 2023. Vitruvius+: An area-efficient RISC-V decoupled vector coprocessor for high performance computing applications. *ACM Transactions on Architecture and Code Optimization* 20, 2, Article 28 (mar 2023), 25 pages. DOI: 10.1145/3575861

[117] Sparsh Mittal, Himanshi Gupta, and Srishti Srivastava. 2021. A survey on hardware security of DNN models and accelerators. *Journal of Systems Architecture* 117 (2021), 102163. DOI: 10.1016/j.sysarc.2021.102163

[118] Gianluca Mittone, Nicoló Tonci, Robert Birke, Iacopo Colonnelli, Doriana Medić, Andrea Bartolini, Roberto Esposito, Emanuele Parisi, Francesco Beneventi, Mirko Polato, Massimo Torquati, Luca Benini, and Marco Aldinucci. 2023. Experimenting with emerging RISC-V systems for decentralised machine learning. In *Proceedings of the 20th ACM International Conference on Computing Frontiers* (Bologna, Italy) *(CF'23)*. Association for Computing Machinery, New York, NY, USA, 73–83. DOI: 10.1145/3587135.3592211

[119] Fabio Montagna, Stefan Mach, Simone Benatti, Angelo Garofalo, Gianmarco Ottavi, Luca Benini, Davide Rossi, and Giuseppe Tagliavini. 2022. A low-power transprecision floating-point cluster for efficient near-sensor data analytics. *IEEE Transactions on Parallel and Distributed Systems* 33, 5 (2022), 1038–1053. DOI: 10.1109/TPDS.2021.3101764

[120] Diksha Moolchandani, Anshul Kumar, and Smruti R. Sarangi. 2021. Accelerating CNN inference on ASICs: A survey. *Journal of Systems Architecture* 113 (2021), 101887. DOI: 10.1016/j.sysarc.2020.101887

[121] M. G. Sarwar Murshed, Christopher Murphy, Daqing Hou, Nazar Khan, Ganesh Ananthanarayanan, and Faraz Hussain. 2021. Machine learning at the network edge: A survey. *ACM Comput. Surv.* 54, 8, Article 170 (oct 2021), 37 pages. DOI: 10.1145/3469029

[122] Vittoriano Muttillo, Luigi Pomante, Marco Santic, and Giacomo Valente. 2023. SystemC-based co-simulation/analysis for system-level hardware/software co-design. *Computers and Electrical Engineering* 110 (2023), 108803.

[123] Davide Nadalini, Manuele Rusci, Giuseppe Tagliavini, Leonardo Ravaglia, Luca Benini, and Francesco Conti. 2022. PULP-TrainLib: Enabling on-device training for RISC-V multi-core MCUs through performance-driven autotuning. In *Proceedings of the Embedded Computer Systems: Architectures, Modeling, and Simulation*, Alex Orailoglu, Marc Reichenbach, and Matthias Jung (Eds.). Springer International Publishing, Cham, 200–216.

[124] Nuno Neves, Joao Mario Domingos, Nuno Roma, Pedro Tomás, and Gabriel Falcao. 2022. Compiling for vector extensions with stream-based specialization. *IEEE Micro* 42, 5 (2022), 49–58. DOI: 10.1109/MM.2022.3173405

[125] Ngo-Doanh Nguyen, Duy-Hieu Bui, and Xuan-Tu Tran. 2022. Tiny neuron network system based on RISC-V processor: A decentralized approach for IoT applications. In *Proceedings of the 2022 International Conference on Advanced Technologies for Communications (ATC)*. 98–103. DOI: 10.1109/ATC55345.2022.9942990

[126] Zhaolong Ning, Hao Hu, Xiaojie Wang, Lei Guo, Song Guo, Guoyin Wang, and Xinbo Gao. 2024. Mobile edge computing and machine learning in the internet of unmanned aerial vehicles: A survey. *ACM Computing Surveys* 56, 1 (January 2024), 1–31. DOI: 10.1145/3604933

[127] Julian Oppermann, Brindusa Mihaela Damian-Kosterhon, Florian Meisel, Tammo Mürmann, Eyck Jentzsch, and Andreas Koch. 2024. Longnail: High-level synthesis of portable custom instruction set extensions for RISC-V processors from descriptions in the open-source CoreDSL language. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3* (La Jolla, CA, USA) *(ASPLOS'24)*. Association for Computing Machinery, New York, NY, USA, 591–606. DOI: 10.1145/3620666.3651375

[128] Gianmarco Ottavi, Angelo Garofalo, Giuseppe Tagliavini, Francesco Conti, Luca Benini, and Davide Rossi. 2020. A mixed-precision RISC-V processor for extreme-edge DNN inference. In *Proceedings of the 2020 IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*. 512–517. DOI: 10.1109/ISVLSI49217.2020.000-5

[129] Gianmarco Ottavi, Angelo Garofalo, Giuseppe Tagliavini, Francesco Conti, Alfio Di Mauro, Luca Benini, and Davide Rossi. 2023. Dustin: A 16-cores parallel ultra-low-power cluster with 2b-to-32b fully flexible bit-precision and vector lockstep execution mode. *IEEE Transactions on Circuits and Systems I: Regular Papers* 70, 6 (2023), 2450–2463. DOI: 10.1109/TCSI.2023.3254810

[130] Vivek Parmar, Ayan Ray, Chithambara Moorthii, Richa Mishra, Deepak Verma, Digamber Pandey, and Manan Suri. 2023. Open-RIMC: Open-source RRAM-based IMC co-processor with reconfigurable logic mapping. In *Proceedings of the 2023 IEEE 23rd International Conference on Nanotechnology (NANO)*. 519–523. DOI: 10.1109/NANO58406.2023.10231247

[131] Kariofyllis Patsidis, Chrysostomos Nicopoulos, Georgios Ch. Sirakoulis, and Giorgos Dimitrakopoulos. 2020. RISC-V2: A scalable RISC-V vector processor. In *Proceedings of the 2020 IEEE International Symposium on Circuits and Systems (ISCAS)*. 1–5. DOI: 10.1109/ISCAS45731.2020.9181071

[132] Matteo Perotti, Matheus Cavalcante, Renzo Andri, Lukas Cavigelli, and Luca Benini. 2024. Ara2: Exploring single-core and multi-core vector processing with an efficient RVV 1.0 compliant open-source processor. *IEEE Transactions on Computers* 73, 7 (2024), 1822–1836. DOI: 10.1109/TC.2024.3388896

[133] Matteo Perotti, Matheus Cavalcante, Alessandro Ottaviano, Jiantao Liu, and Luca Benini. 2023. Yun: An open-source, 64-bit RISC-V-based vector processor with multi-precision integer and floating-point support in 65-nm CMOS. *IEEE Transactions on Circuits and Systems II: Express Briefs* 70, 10 (2023), 3732–3736. DOI: 10.1109/TCSII.2023.3292579

[134] Daniel Pestana, Pedro R. Miranda, João D. Lopes, Rui P. Duarte, Mário P. Véstias, Horácio C. Neto, and José T. De Sousa. 2021. A full featured configurable accelerator for object detection with YOLO. *IEEE Access* 9 (2021), 75864–75877. DOI: 10.1109/ACCESS.2021.3081818

[135] Thi-Nhan Pham, Quang-Kien Trinh, Ik-Joon Chang, and Massimo Alioto. 2022. STT-BNN: A novel STT-MRAM in-memory computing macro for binary neural networks. *IEEE Journal on Emerging and Selected Topics in Circuits and Systems* 12, 2 (2022), 569–579. DOI: 10.1109/JETCAS.2022.3169759

[136] Michele Piccoli, Davide Zoni, William Fornaciari, Giuseppe Massari, Marco Cococcioni, Federico Rossi, Sergio Saponara, and Emanuele Ruffaldi. 2023. Dynamic power consumption of the full posit processing unit: Analysis and experiments. In *Proceedings of the 14th Workshop on Parallel Programming and Run-Time Management Techniques for Many-Core Architectures and 12th Workshop on Design Tools and Architectures for Multicore Embedded Computing Platforms (PARMA-DITAM 2023) (Open Access Series in Informatics (OASIcs), Vol. 107)*, João Bispo, Henri-Pierre Charles, Stefano Cherubin, and Giuseppe Massari (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 6:1–6:11. DOI: 10.4230/OASIcs.PARMA-DITAM.2023.6

[137] Michael Platzer and Peter Puschner. 2021. Vicuna: A timing-predictable RISC-V vector coprocessor for scalable parallel computation. In *Proceedings of the 33rd Euromicro Conference on Real-Time Systems (ECRTS 2021) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 196)*, Björn B. Brandenburg (Ed.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 1:1–1:18. DOI: 10.4230/LIPIcs.ECRTS.2021.1

[138] Ross Porter, Sam Morgan, and Morteza Biglari-Abhari. 2019. Extending a soft-core RISC-V processor to accelerate CNN inference. In *Proceedings of the 2019 International Conference on Computational Science and Computational Intelligence (CSCI)*. 694–697. DOI: 10.1109/CSCI49370.2019.00130

[139] Kartik Prabhu, Albert Gural, Zainab F. Khan, Robert M. Radway, Massimo Giordano, Kalhan Koul, Rohan Doshi, John W. Kustin, Timothy Liu, Gregorio B. Lopes, Victor Turbiner, et al. 2022. CHIMERA: A 0.92-TOPS, 2.2-TOPS/W Edge AI accelerator with 2-MByte on-chip foundry resistive RAM for efficient training and inference. *IEEE Journal of Solid-State Circuits* 57, 4 (2022), 1013–1026. DOI: 10.1109/JSSC.2022.3140753

[140] Arpan Suravi Prasad, Moritz Scherer, Francesco Conti, Davide Rossi, Alfio Di Mauro, Manuel Eggimann, Jorge Tomás Gómez, Ziyun Li, Syed Shakib Sarwar, Zhao Wang, Barbara De Salvo, and Luca Benini. 2024. Siracusa: A 16 nm heterogenous RISC-V SoC for extended reality with At-MRAM neural engine. *IEEE Journal of Solid-State Circuits* 59, 7 (2024), 2055–2069. DOI: 10.1109/JSSC.2024.3385987

[141] Samuel Riedel, Gua Hao Khov, Sergio Mazzola, Matheus Cavalcante, Renzo Andri, and Luca Benini. 2023. MemPool meets systolic: Flexible systolic computation in a large shared-memory processor cluster. In *Proceedings of the 2023 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. 1–2. DOI: 10.23919/DATE56975.2023.10136909

[142] Risikesh RK, Sharad Sinha, and Nanditha Rao. 2021. Variable bit-precision vector extension for RISC-V based processors. In *Proceedings of the 2021 IEEE 14th International Symposium on Embedded Multicore/Many-core Systems-on-Chip (MCSoC)*. 114–121. DOI: 10.1109/MCSoC51149.2021.00024

[143] Michael Rogenmoser, Yvan Tortorella, Davide Rossi, Francesco Conti, and Luca Benini. 2025. Hybrid modular redundancy: Exploring modular redundancy approaches in RISC-V multi-core computing clusters for reliable processing in space. *ACM Transactions on Cyber-Physical Systems* 9, 1, Article 8 (Jan. 2025), 29 pages. DOI: 10.1145/3635161

[144] Muhammad Sabih, Abrarul Karim, Jakob Wittmann, Frank Hannig, and Jürgen Teich. 2024. Hardware/software co-design of RISC-V extensions for accelerating sparse DNNs on FPGAs. In *Proceedings of the 2024 International Conference on Field Programmable Technology (ICFPT)*. 01–09. DOI: 10.1109/ICFPT64416.2024.11113397

[145] Alejandra Sanchez-Flores, Lluc Alvarez, and Bartomeu Alorda-Ladaria. 2022. A review of CNN accelerators for embedded systems based on RISC-V. In *Proceedings of the 2022 IEEE International Conference on Omni-layer Intelligent Systems (COINS)*. 1–6. DOI: 10.1109/COINS54846.2022.9855006

[146] Marcel Sarraseca, Sergi Alcaide, Francisco Fuentes, Juan Carlos Rodriguez, Feng Chang, Ilham Lasfar, Ramon Canal, Francisco J. Cazorla, and Jaume Abella. 2023. SafeLS: An open source implementation of a lockstep NOEL-V RISC-V

core. In *Proceedings of the 2023 IEEE 29th International Symposium on On-Line Testing and Robust System Design (IOLTS)*. 1–7. DOI: 10.1109/IOLTS59296.2023.10224867

[147] Mahadev Satyanarayanan. 2017. The emergence of edge computing. *Computer* 50, 1 (2017), 30–39. DOI: 10.1109/MC.2017.9

[148] Paul Scheffler, Florian Zaruba, Fabian Schuiki, Torsten Hoefler, and Luca Benini. 2023. Sparse stream semantic registers: A lightweight ISA extension accelerating general sparse linear algebra. *IEEE Transactions on Parallel and Distributed Systems* 34, 12 (2023), 3147–3161. DOI: 10.1109/TPDS.2023.3322029

[149] Moritz Scherer, Alfio Di Mauro, Georg Rutishauser, Tim Fischer, and Luca Benini. 2022. A 1036 TOp/s/W, 12.2 mW, 2.72 uJ/inference all digital TNN accelerator in 22 nm FDX technology for TinyML applications. In *Proceedings of the 2022 IEEE Symposium in Low-Power and High-Speed Chips (COOL CHIPS)*. 1–3. DOI: 10.1109/COOLCHIPS54332.2022.9772668

[150] Moritz Scherer, Alfio Di Mauro, Tim Fischer, Georg Rutishauser, and Luca Benini. 2023. TCN-CUTIE: A 1,036-TOp/s/W, 2.72-µJ/Inference, 12.2-mW all-digital ternary accelerator in 22-nm FDX technology. *IEEE Micro* 43, 1 (2023), 42–48. DOI: 10.1109/MM.2022.3226630

[151] Jürgen Schmidhuber. 2015. Deep learning in neural networks: An overview. *Neural Networks* 61 (2015), 85–117. DOI: 10.1016/j.neunet.2014.09.003

[152] Colin Schmidt, John Wright, Zhongkai Wang, Eric Chang, Albert Ou, Woorham Bae, Sean Huang, Vladimir Milovanović, Anita Flynn, Brian Richards, Krste Asanović, Elad Alon, and Borivoje Nikolić. 2022. An eight-core 1.44-GHz RISC-V vector processor in 16-nm FinFET. *IEEE Journal of Solid-State Circuits* 57, 1 (2022), 140–152. DOI: 10.1109/JSSC.2021.3118046

[153] Fabian Schuiki, Florian Zaruba, Torsten Hoefler, and Luca Benini. 2021. Stream semantic registers: A lightweight RISC-V ISA extension achieving full compute utilization in single-issue cores. *IEEE Transactions on Computers* 70, 2 (2021), 212–227. DOI: 10.1109/TC.2020.2987314

[154] Niraj N. Sharma, Riya Jain, Mohana Madhumita Pokkuluri, Sachin B. Patkar, Rainer Leupers, Rishiyur S. Nikhil, and Farhad Merchant. 2023. CLARINET: A quire-enabled RISC-V-based framework for posit arithmetic empiricism. *Journal of Systems Architecture* 135 (2023), 102801. DOI: 10.1016/j.sysarc.2022.102801

[155] Dinggang Shen, Guorong Wu, and Heung-Il Suk. 2017. Deep learning in medical image analysis. *Annual Review of Biomedical Engineering* 19, Volume 19, 2017 (2017), 221–248. DOI: 10.1146/annurev-bioeng-071516-044442

[156] Runbin Shi, Junjie Liu, Hayden K.-H. So, Shuo Wang, and Yun Liang. 2019. E-LSTM: Efficient inference of sparse LSTM on embedded heterogeneous system. In *Proceedings of the 56th Annual Design Automation Conference 2019* (Las Vegas, NV, USA) *(DAC'19)*. Association for Computing Machinery, New York, NY, USA, Article 182, 6 pages. DOI: 10.1145/3316781.3317813

[157] Md. Maruf Hossain Shuvo, Syed Kamrul Islam, Jianlin Cheng, and Bashir I. Morshed. 2023. Efficient acceleration of deep learning inference on resource-constrained edge devices: A review. *Proceedings of the IEEE* 111, 1 (2023), 42–91. DOI: 10.1109/JPROC.2022.3226481

[158] Cristina Silvano, Daniele Ielmini, Fabrizio Ferrandi, Leandro Fiorin, Serena Curzel, Luca Benini, Francesco Conti, Angelo Garofalo, Cristian Zambelli, Enrico Calore, et al. 2025. A survey on deep learning hardware accelerators for heterogeneous HPC platforms. *ACM Computing Surveys* 57, 11, Article 286 (June 2025), 39 pages. DOI: 10.1145/3729215

[159] IEEE Computer Society. 2019. IEEE Standard for Floating-Point Arithmetic. DOI: https://doi.org/10.1109/IEEESTD.2019.8766229

[160] Taha Soliman, Swetaki Chatterjee, Nellie Laleni, Franz Müller, Tobias Kirchner, Norbert Wehn, Thomas Kämpfe, Yogesh Singh Chauhan, and Hussam Amrouch. 2023. First demonstration of in-memory computing crossbar using multi-level Cell FeFET. *Nature Communications* 14, 1 (2023), 6348.

[161] Víctor Soria-Pardos, Max Doblas, Guillem López–Paradís, Gerard Candón, Narcís Rodas, Xavier Carril, Pau Fontova–Musté, Neiel Leyva, Santiago Marco-Sola, and Miquel Moretó. 2022. Sargantana: A 1 GHz+ In-Order RISC-V processor with SIMD vector extensions in 22nm FD-SOI. In *Proceedings of the 2022 25th Euromicro Conference on Digital System Design (DSD)*. 254–261. DOI: 10.1109/DSD57027.2022.00042

[162] Rafael Sousa, Marcio Pereira, Yongin Kwon, Taeho Kim, Namsoon Jung, Chang Soo Kim, Michael Frank, and Guido Araujo. 2023. Tensor slicing and optimization for multicore NPUs. *Journal of Parallel and Distributed Computing* 175 (2023), 66–79. DOI: 10.1016/j.jpdc.2022.12.008

[163] SpinalHDL. 2017. VexRiscv. Retrieved from https://github.com/SpinalHDL/VexRiscv

[164] Vivienne Sze, Yu-Hsin Chen, Tien-Ju Yang, and Joel S. Emer. 2017. Efficient processing of deep neural networks: A tutorial and survey. *Proceedings of the IEEE* 105, 12 (2017), 2295–2329. DOI: 10.1109/JPROC.2017.2761740

[165] Tuan Ta, Khalid Al-Hawaj, Nick Cebry, Yanghui Ou, Eric Hall, Courtney Golden, and Christopher Batten. 2022. big.VLITTLE: On-demand data-parallel acceleration for mobile systems on chip. In *Proceedings of the 2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 181–198. DOI: 10.1109/MICRO56248.2022.00025

[166] Wenkai Tang and Peiyong Zhang. 2022. GPGCN: A general-purpose graph convolution neural network accelerator based on RISC-V ISA extension. *Electronics* 11, 22 (2022). DOI: 10.3390/electronics11223833

[167] TensorFlow. 2015. MLIR dialects and utilities for TensorFlow, TensorFlow Lite and XLA. Retrieved from https://github.com/tensorflow/tensorflow/tree/master/tensorflow/compiler/mlir

[168] TensorFlow. 2023. TensorFlow Lite for Microcontrollers. Retrieved from https://github.com/tensorflow/tflite-micro

[169] Sugandha Tiwari, Neel Gala, Chester Rebeiro, and V. Kamakoti. 2021. PERI: A configurable posit enabled RISC-V core. *ACM Transactions on Architecture and Code Optimization* 18, 3, Article 25 (apr 2021), 26 pages. DOI: 10.1145/3446210

[170] Markus Ulbricht, Yvan Tortorella, Michael Rogenmoser, Li Lu, Junchao Chen, Francesco Conti, Milos Krstic, and Luca Benini. 2023. PULP fiction no more—dependable PULP systems for space. In *Proceedings of the 2023 IEEE European Test Symposium (ETS)*. 1–10. DOI: 10.1109/ETS56758.2023.10174164

[171] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Proceedings of the Advances in Neural Information Processing Systems*, I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett (Eds.), Vol. 30. Curran Associates, Inc. Retrieved from https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf

[172] Anu Verma, Priyamvada Sharma, and Bishnu Prasad Das. 2022. RISC-V core with approximate multiplier for error-tolerant applications. In *Proceedings of the 2022 25th Euromicro Conference on Digital System Design (DSD)*. 239–246. DOI: 10.1109/DSD57027.2022.00040

[173] Vaibhav Verma and Mircea R. Stan. 2022. AI-PiM—extending the RISC-V processor with processing-in-memory functional units for AI inference at the edge of IoT. *Frontiers in Electronics* 3 (2022), 1–15. DOI: 10.3389/felec.2022.898273

[174] Vaibhav Verma, Tommy Tracy II, and Mircea R. Stan. 2022. EXTREM-EDGE—EXtensions To RISC-V for energy-efficient ML inference at the EDGE of IoT. *Sustainable Computing: Informatics and Systems* 35 (2022), 100742. DOI: 10.1016/j.suscom.2022.100742

[175] Erwei Wang, James J. Davis, Ruizhe Zhao, Ho-Cheung Ng, Xinyu Niu, Wayne Luk, Peter Y. K. Cheung, and George A. Constantinides. 2019. Deep neural network approximation for custom hardware: Where we've been, where we're going. *ACM Computing Surveys* 52, 2, Article 40 (may 2019), 39 pages. DOI: 10.1145/3309551

[176] Hansen Wang, Dongju Li, and Tsuyoshi Isshiki. 2023. Reconfigurable CNN accelerator embedded in instruction extended RISC-V core. In *Proceedings of the 2023 6th International Conference on Electronics Technology (ICET)*. 945–954. DOI: 10.1109/ICET58434.2023.10211513

[177] Shihang Wang, Jianghan Zhu, Qi Wang, Can He, and Terry Tao Ye. 2021. Customized instruction on RISC-V for winograd-based convolution acceleration. In *Proceedings of the 2021 IEEE 32nd International Conference on Application-specific Systems, Architectures and Processors (ASAP)*. 65–68. DOI: 10.1109/ASAP52443.2021.00018

[178] Yipeng Wang, Mengtian Yang, Chieh-Pu Lo, and Jaydeep P. Kulkarni. 2024. 30.6 Vecim: A 289.13GOPS/W RISC-V vector co-processor with compute-in-memory vector register file for efficient high-performance computing. In *Proceedings of the 2024 IEEE International Solid-State Circuits Conference (ISSCC)*, Vol. 67. 492–494. DOI: 10.1109/ISSCC49657.2024.10454387

[179] Nils-Johan Wessman, Fabio Malatesta, Jan Andersson, Paco Gomez, Miguel Masmano, Vicente Nicolau, Jimmy Le Rhun, Guillem Cabo, Francisco Bas, Ruben Lorenzo, Oriol Sala, David Trilla, and Jaume Abella. 2021. De-RISC: The first RISC-V space-grade platform for safety-critical systems. In *Proceedings of the 2021 IEEE Space Computing Conference (SCC)*. 17–26. DOI: 10.1109/SCC49971.2021.00010

[180] James Hardy Wilkinson. 2023. *Rounding Errors in Algebraic Processes*. SIAM.

[181] S. Winograd. 1971. On multiplication of 2 × 2 matrices. *Linear Algebra and its Applications* 4, 4 (1971), 381–388. DOI: 10.1016/0024-3795(71)90009-7

[182] Carole-Jean Wu, David Brooks, Kevin Chen, Douglas Chen, Sy Choudhury, Marat Dukhan, Kim Hazelwood, Eldad Isaac, Yangqing Jia, Bill Jia, et al. 2019. Machine learning at facebook: Understanding inference at the edge. In *Proceedings of the 2019 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 331–344. DOI: 10.1109/HPCA.2019.00048

[183] Rui Xu, Sheng Ma, Yang Guo, and Dongsheng Li. 2023. A survey of design and optimization for systolic array-based DNN accelerators. *ACM Computing Surveys* 56, 1, Article 20 (aug 2023), 37 pages. DOI: 10.1145/3604802

[184] Chun-Chieh Yang, Yi-Ru Chen, Hui-Hsin Liao, Yuan-Ming Chang, and Jenq-Kuen Lee. 2023. Auto-tuning fixed-point precision with TVM on RISC-V packed SIMD extension. *ACM Transactions on Design Automation of Electronic Systems* 28, 3, Article 33 (mar 2023), 21 pages. DOI: 10.1145/3569939

[185] Yosys Headquarters. 2015. PicoRV32 - A Size-Optimized RISC-V CPU. Retrieved from https://github.com/YosysHQ/picorv32

[186] Meng-Shiun Yu, Chuan-Yue Yuan, Tai-Liang Chen, and Jenq-Kuen Lee. 2024. Case study: Optimization methods with TVM hybrid-op on RISC-V packed SIMD. *IEEE Access* 12 (2024), 64193–64211. DOI: 10.1109/ACCESS.2024.3397195

[187] Florian Zaruba and Luca Benini. 2019. The cost of application-class processing: Energy and performance analysis of a linux-ready 1.7-GHz 64-Bit RISC-V core in 22-nm FDSOI technology. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 27, 11 (2019), 2629–2640. DOI: 10.1109/TVLSI.2019.2926114

[188] Florian Zaruba, Fabian Schuiki, Torsten Hoefler, and Luca Benini. 2021. Snitch: A tiny pseudo dual-issue processor for area and energy efficient execution of floating-point intensive workloads. *IEEE Transactions on Computers* 70, 11 (2021), 1845–1860. DOI: 10.1109/TC.2020.3027900

[189] Hongbin Zhang, Mingjie Xing, Yanjun Wu, and Chen Zhao. 2023. Compiler technologies in deep learning co-design: A survey. *Intelligent Computing* 2 (2023), 0040.

[190] Jerry Zhao, Ben Korpan, Abraham Gonzalez, and Krste Asanovic. 2020. SonicBOOM: The 3rd generation berkeley out-of-order machine. In *Proceedings of the 4th Workshop on Computer Architecture Research with RISC-V*, Vol. 5. 1–7.

[191] Xiaotian Zhao, Ruge Xu, Yimin Gao, Vaibhav Verma, Mircea R. Stan, and Xinfei Guo. 2024. Edge-MPQ: Layer-wise mixed-precision quantization with tightly integrated versatile inference units for edge computing. *IEEE Transactions on Computers* 73, 11 (2024), 2504–2519. DOI: 10.1109/TC.2024.3441860

[192] Houji Zhou, Zhiwei Zhou, Shengguang Ren, Jia Chen, Yi Li, and Xiangshui Miao. 2024. Heterogeneous instruction set architecture for RRAM-enabled in-memory computing. In *Proceedings of the 18th ACM International Symposium on Nanoscale Architectures* (Dresden, Germany) *(NANOARCH'23)*. Association for Computing Machinery, New York, NY, USA, Article 7, 6 pages. DOI: 10.1145/3611315.3633244

[193] Brian Zimmer, Yunsup Lee, Alberto Puggelli, Jaehwa Kwak, Ruzica Jevtic, Ben Keller, Stevo Bailey, Milovan Blagojevic, Pi-Feng Chiu, Hanh-Phuc Le, et al. 2015. A RISC-V vector processor with tightly-integrated switched-capacitor DC-DC converters in 28nm FDSOI. In *Proceedings of the 2015 Symposium on VLSI Circuits (VLSI Circuits)*. C316–C317. DOI: 10.1109/VLSIC.2015.7231305

[194] Davide Zoni and Andrea Galimberti. 2022. Cost-effective fixed-point hardware support for RISC-V embedded systems. *Journal of Systems Architecture* 126 (2022), 102476. DOI: 10.1016/j.sysarc.2022.102476

[195] Davide Zoni, Andrea Galimberti, and William Fornaciari. 2021. An FPU design template to optimize the accuracy-efficiency-area trade-off. *Sustainable Computing: Informatics and Systems* 29 (2021), 100450. DOI: 10.1016/j.suscom.2020.100450

[196] Davide Zoni, Andrea Galimberti, and William Fornaciari. 2023. A survey on run-time power monitors at the edge. *ACM Computing Surveys* 55, 14s, Article 325 (Jul 2023), 33 pages. DOI: 10.1145/3593044

[197] Davide Zoni, Andrea Galimberti, and Davide Galli. 2025. An FPGA-based open-source hardware-software framework for side-channel security research. *IEEE Transactions on Computers* 74, 6 (2025), 2087–2100. DOI: 10.1109/TC.2025.3551936

Appendices

A Background on RISC-V Instruction Set Architecture

In this appendix, we describe the most common extensions that compose the RISC-V ISA and briefly discuss the naming conventions adopted for RISC-V architectures defined by the standard's specification and also used in the rest of this manuscript.

A.1 Main Extensions

The *I base integer* instruction set implements integer computational instructions, integer loads and stores, and control-flow instructions, while the *M integer multiplication and division* extension adds instructions to multiply and divide values in the integer registers and the *A atomic* extension includes instructions that atomically read, modify, and write memory for inter-processor synchronization.

The *C compressed* instruction extension introduces narrower 16-bit versions of the most common 32-bit instructions to reduce the code size in highly constrained platforms such as MCUs.

The *F single-precision FP* extension adds 32-bit FP registers, single-precision computational instructions, and single-precision loads and stores, while the *D double-precision FP* extension expands the FP registers to a 64-bit width and adds double-precision computational instructions, loads, and stores. Both extensions comply with the IEEE 754 technical standard for FP arithmetic [159].

The *V vector* extension provides a flexible vector processing architecture that supports variable-length vector operations, enabling efficient data-level parallelism and scalable performance across different applications. It supports a wide range of vector operations, including arithmetic, logical, and memory access instructions. Vector length can be configured dynamically, allowing optimization for various workloads, and different hardware implementations can vary in the number of vector lanes, enabling customization and scalability based on performance and resource requirements.

The *P packed-SIMD* extension enables the parallel processing of multiple small data elements that fit in a single register through a single instruction and without any significant increase in the usage of hardware resources. The most notable instructions, targeting DSP applications and highly quantized ML ones, include arithmetic additions and multiplications as well as bit-manipulation ones for the efficient handling of packed data.

A.2 Naming Convention

RISC-V architectures are described by concatenating the base integer prefix, i.e., RV followed by a number corresponding to the address space size in bits for the base integer ISA, with the names of the included extensions. For example, RV64IMF denotes a 64-bit RISC-V architecture with support for the I, M, and F extensions. A special notation abbreviates IMAFD with a G, e.g., RV32GC architectures are 32-bit ones that support I, M, A, F, D, and C extensions.

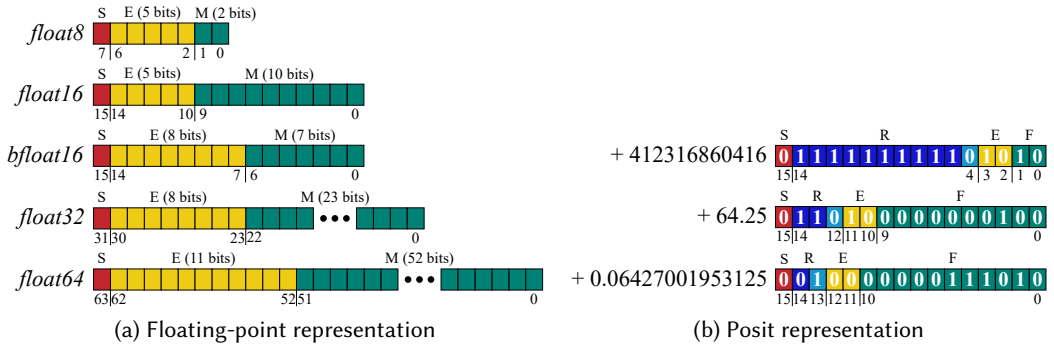


Fig. 3. Floating-point and Posit number system representations: (a) IEEE 754 and other common floating-point data formats and (b) examples of real values encoded in 16-bit Posit format with 2-bit exponent size.

B Background on Data Representation

In this appendix, we discuss the most widely adopted number systems, including both traditional fixed-point and IEEE 754 FP representations, as well as more modern proposals ranging from FP formats specifically designed for DL to Posit.

B.1 Fixed-Point Representation

Fixed-point representation stores numbers with a fixed number of digits after the radix point and offers simple arithmetic operations that can leverage the traditional integer computing pipeline and are, therefore, well suited to adoption and implementation in embedded systems. Its range and precision are limited, making it unsuitable for very large or very small numbers. However, DL applications often use values close to 1, i.e., neither large nor small, and are resilient to approximations, thus favoring the adoption of the fixed-point representation for such workloads. Fixed-point values are represented by treating their binary encoding sequence as a fixed integer part and a fixed fractional one, e.g., in an `8.8` signed fixed-point format, `01000000_01000000` represents a +64.25 real value.

B.2 Floating-Point Representation

FP representation uses a sign (S in Figure 3(a)), an exponent (E), and a mantissa (M) to represent a wide range of values with varying precision, thus making its implementation more complex and costly than the fixed-point one. FP values are represented according to the equation $(-1)^S \times 2^{E-B} \times 1.\{M\}$, e.g., a +64.25 real value is encoded as `0_10000101_00000001000000000000000` as a

32-bit IEEE 754 standard single-precision number with a 1-bit sign, an 8-bit exponent, and a 23-bit significand and with a bias (B) value of 127.

FP representation is dominated by the IEEE 754 standard [159], which defines single-precision (*float32* in Figure 3(a)) and double-precision (*float64*) formats. These formats balance range, precision, and compatibility but are resource-intensive to implement. Reduced-precision variants using fewer bits for the exponent and mantissa have therefore emerged to address domain-specific needs and reduce storage and computational costs. For example, IEEE 754 also defines a half-precision format (*float16*), while 8-bit formats (*float8*) have been proposed as an extension to the standard and are increasingly used in the open literature for AI workloads. The non-IEEE *bfloat16* format was notably developed specifically for DL purposes and retains the same 8-bit exponent as *float32* but reduces the mantissa to 7 bits, sacrificing precision while maintaining dynamic range.

B.3 Posit Representation

Posit [69] representation uses a variable-length format to achieve better precision and dynamic range than IEEE 754 while aiming for higher accuracy and efficiency, particularly for values close to 1, making it useful for scientific computing and ML applications. Its avoidance of special cases such as infinity and NaN further reduces complexity compared to the IEEE 754 FP representation. Posit values are decoded by extracting the sign (S in Figure 3(b)), regime (R), exponent (E), and fraction (F) fields from their representation, e.g., the binary value `0_110_10_0000000100` encoding a 16-bit Posit with exponent size *es* of 2 is interpreted as follows:

(1) *sign* - the leftmost bit is `0`, indicating a positive number;
(2) *regime* - the next bits until a bit flip, `110`, determine the regime, whose value is equal to the number of leading identical bits minus 1 if the leading bits are 1s and to the negative of the number of leading identical bits if they are `0`s, thus in this case the regime value is equal to $K = 2 - 1 = 1$;
(3) *exponent* - since $es = 2$, the two bits right of the regime field correspond to the exponent, i.e., the `10` bits stand for an exponent of 2;
(4) *fraction* - the remaining bits `0000000100` correspond to a 0.00390625 fraction.

The Posit value is then computed as $(-1)^S \cdot useed^K \cdot 2^E \cdot (1 + F)$, where $useed = 2^{2^{es}}$, which in the current example results in $(-1)^0 \cdot 2^4 \cdot 2^2 \cdot (1 + 0.00390625)$, that is, equal to +64.25.

Received 8 July 2024; revised 26 September 2025; accepted 13 October 2025